

Lecture 18

Convolutional Neural Network

Introduction to convolutional architecture for image analysis

EECS 189/289, Fall 2025 @ UC Berkeley
Joseph E. Gonzalez and Narges Norouzi

Roadmap

- Network Design
Considerations for Images
- Element of the
Convolutional Neural
Network
- CNN Design
- CNN History



2499499

Network Design Considerations for Images

- Network Design
Considerations for Images

- Element of the
Convolutional Neural
Network
- CNN Design
- CNN History

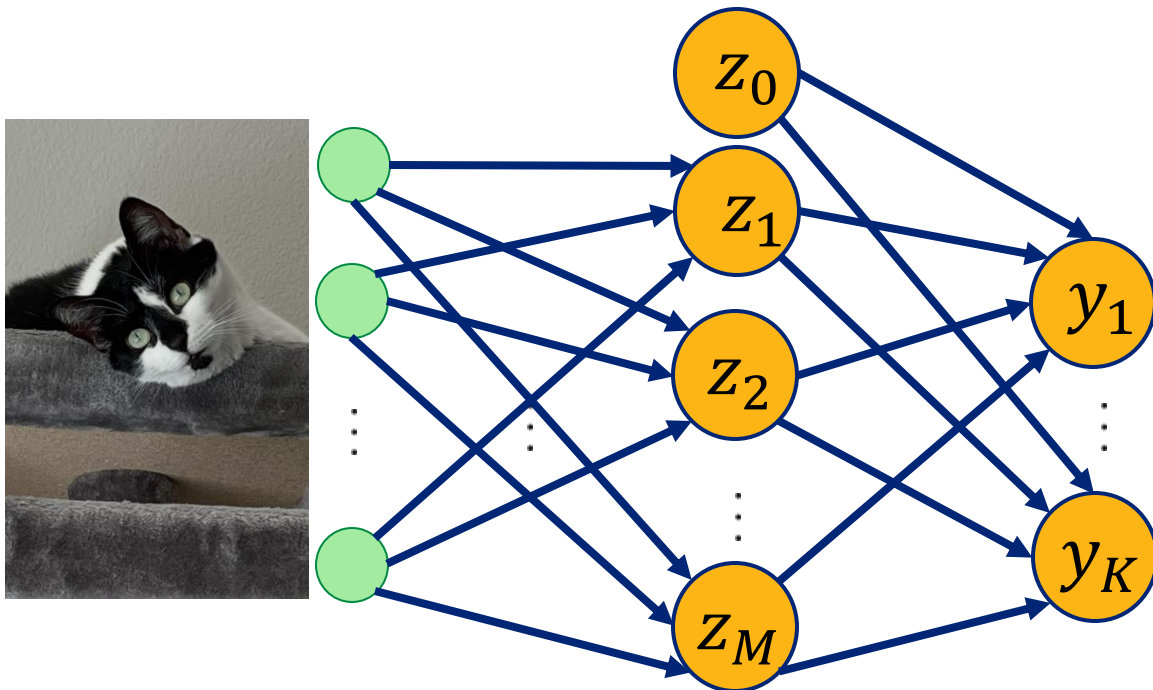


2499499

Story So Far



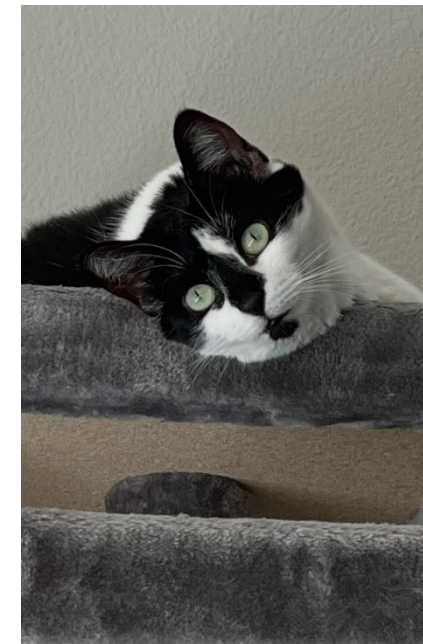
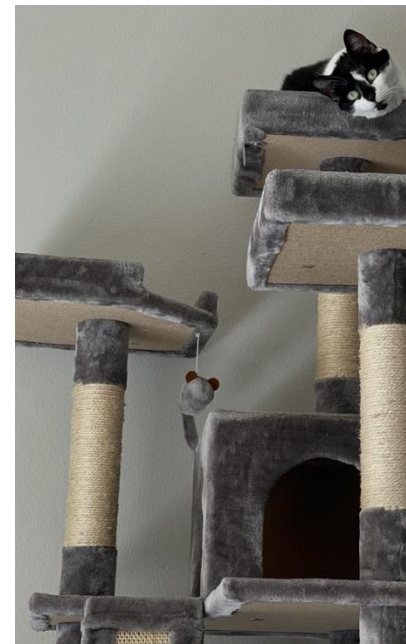
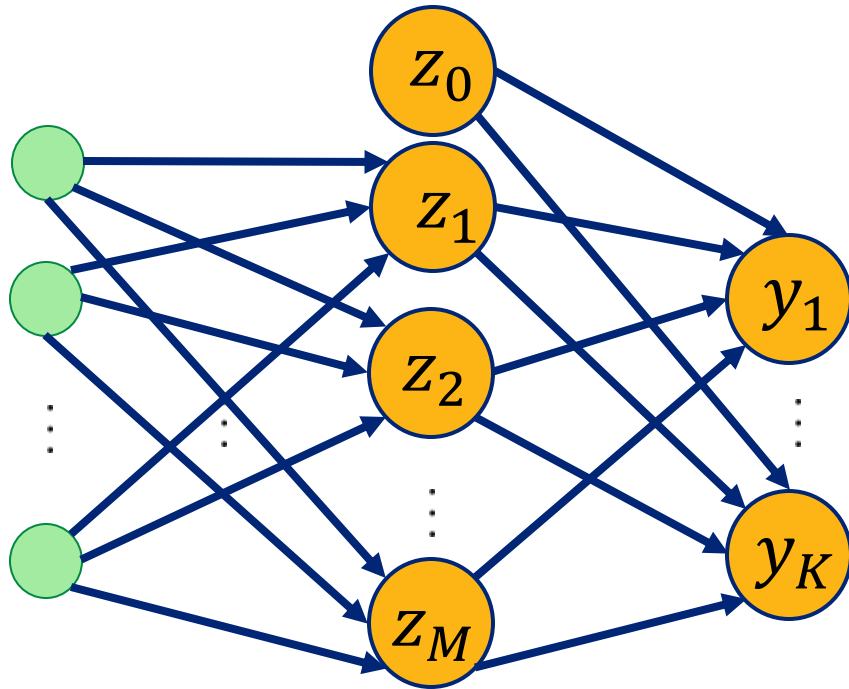
- NNs are universal function approximators.
- NNs can be trained through variations of gradient descent.
- Can recognize patterns in data.



A Problem



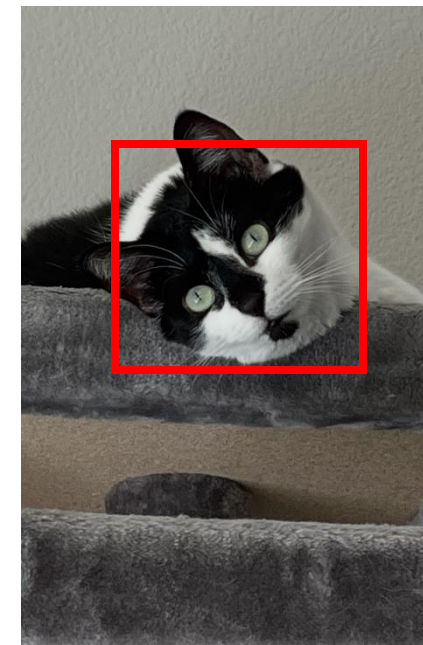
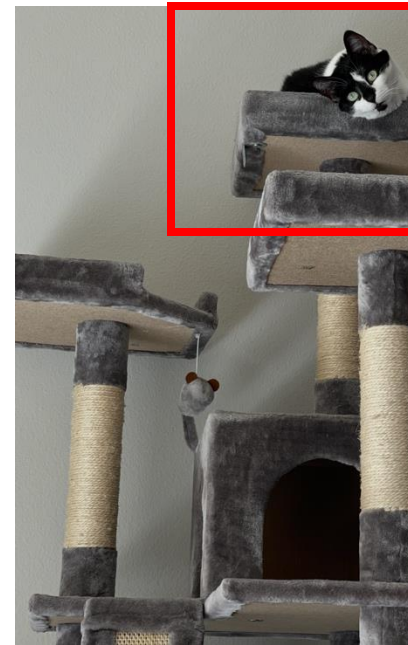
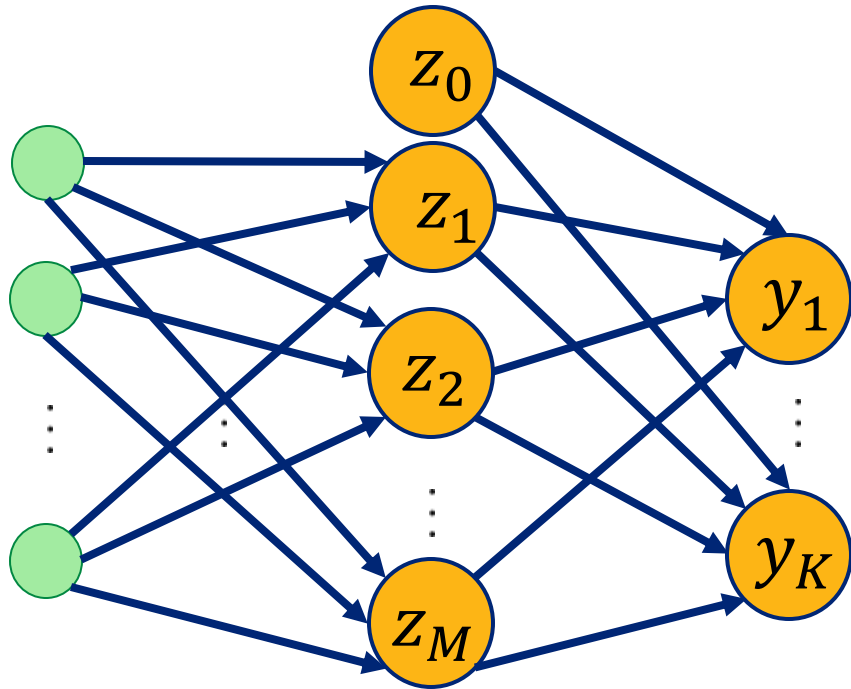
- Will a NN that recognizes the left image as a cat also recognize the one on the right one as a cat?



A Problem



- Will a NN that recognizes the left image as a cat also recognize the one on the right one as a cat?



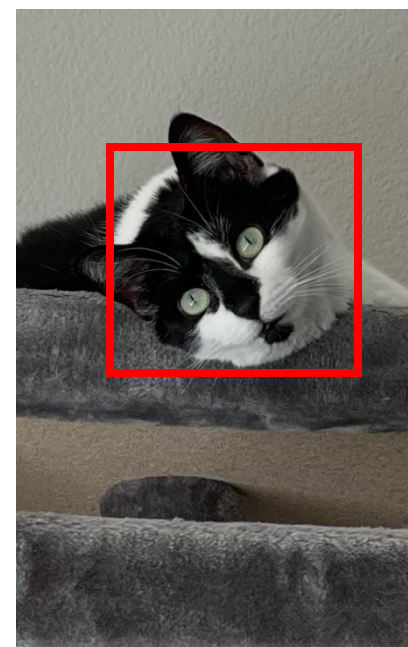
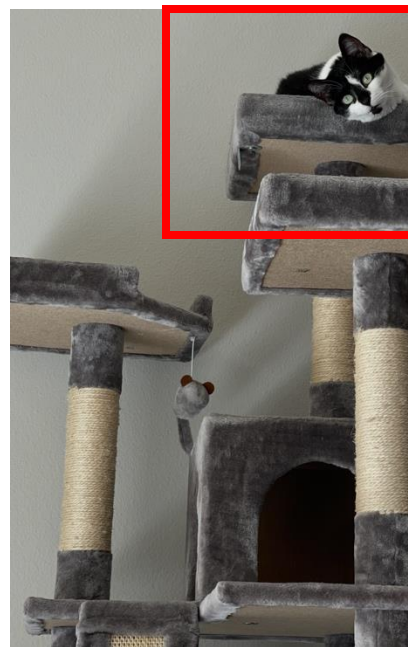
A network that recognizes "cat" feature regardless of the location and scale of the target object.

A Problem



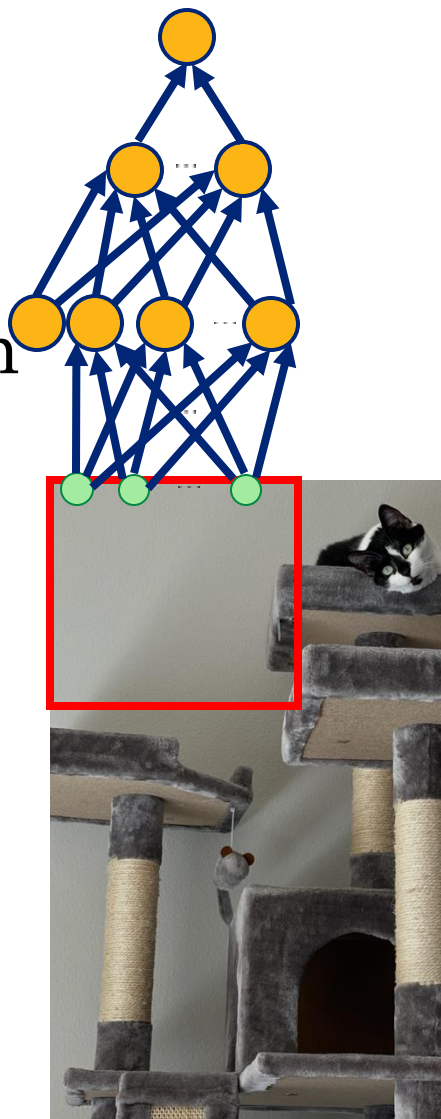
2499499

**Network must be shift
invariant.**



Solution: Scanning

- Scan for the presence of the cat in the image.

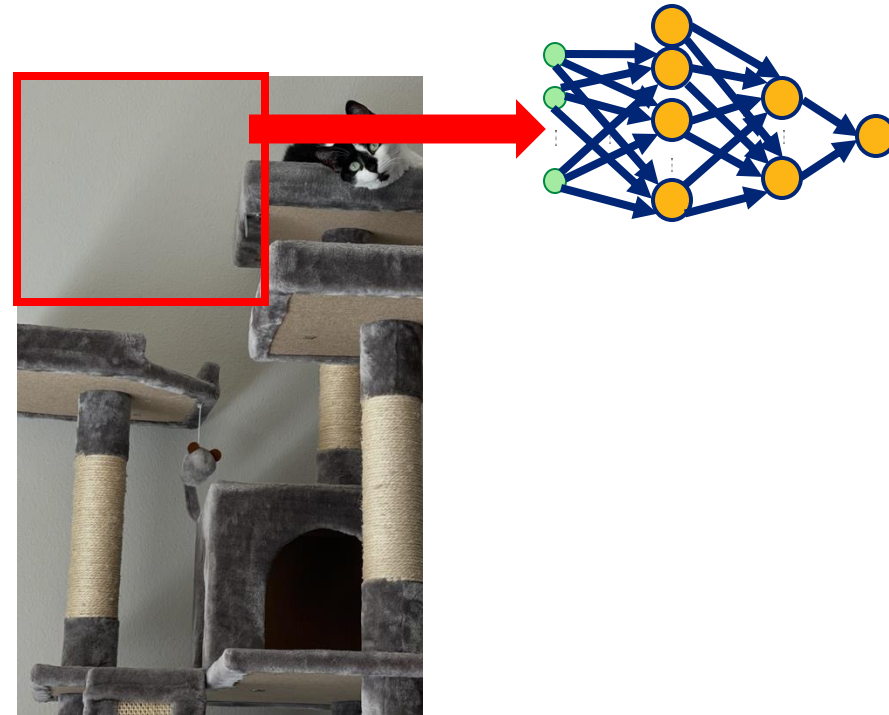


2499499

Scanning Is Referred to as Convolution



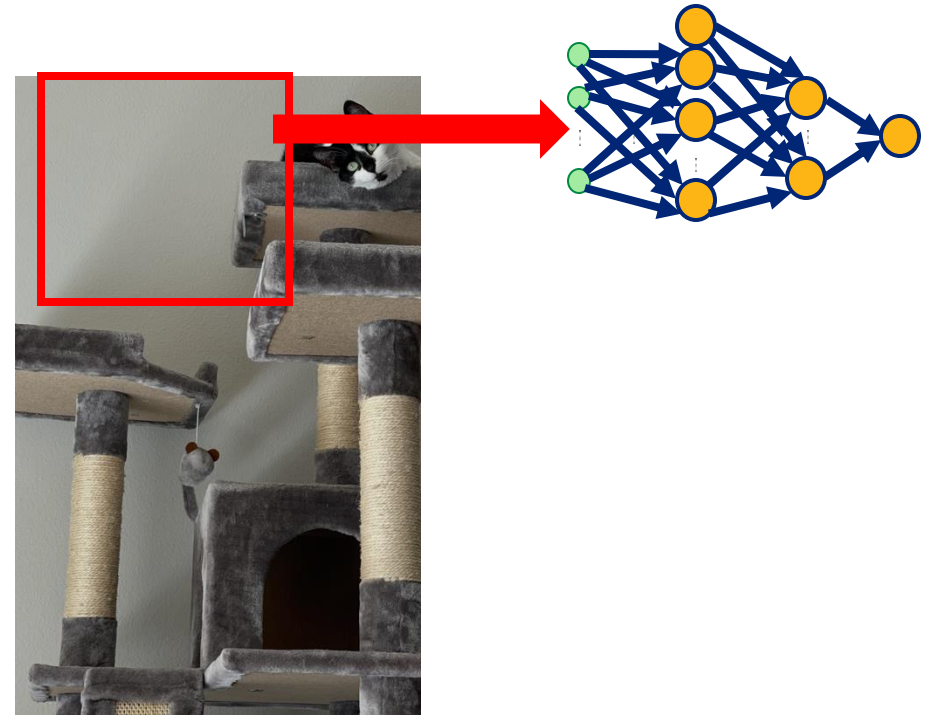
- We need to apply a feature extractor in each region of the image.



Scanning Is Referred to as Convolution



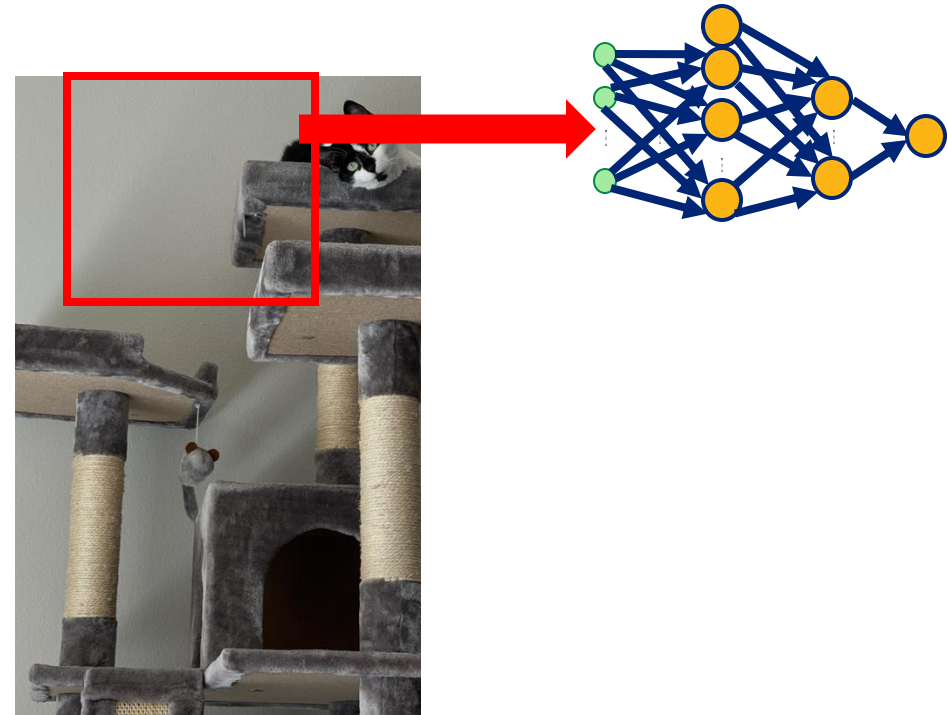
- We need to apply a feature extractor in each region of the image.



Scanning Is Referred to as Convolution



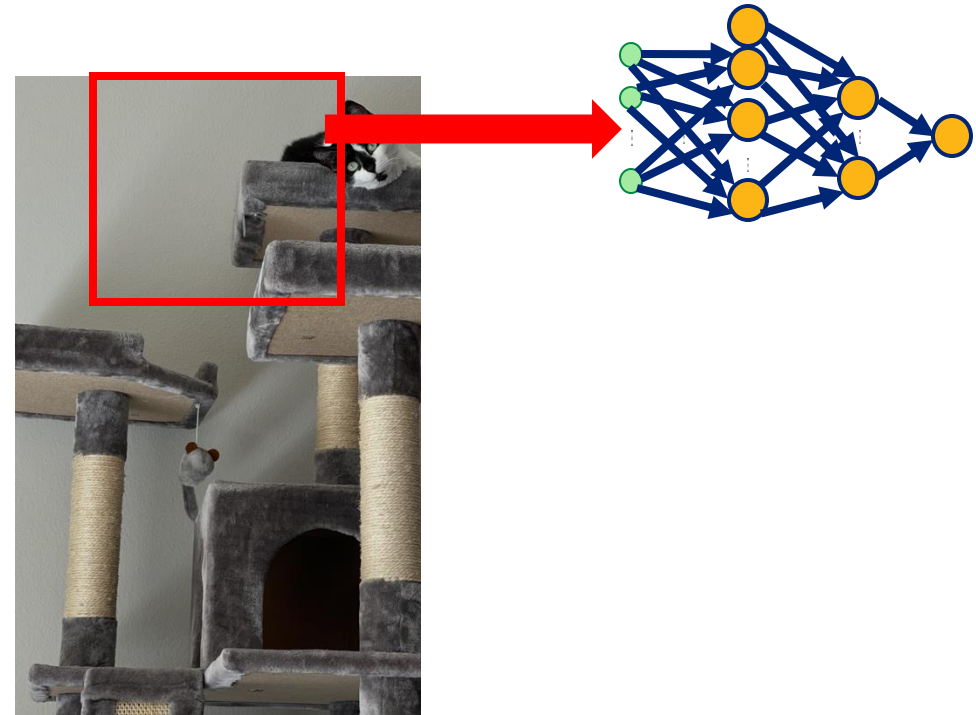
- We need to apply a feature extractor in each region of the image.



Scanning Is Referred to as Convolution



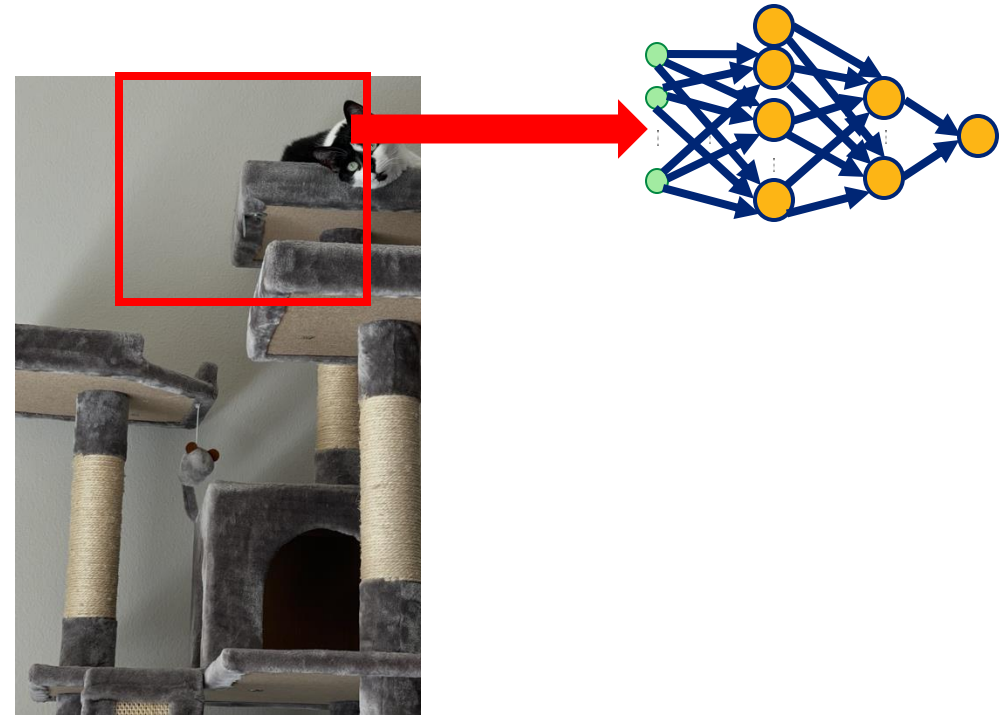
- We need to apply a feature extractor in each region of the image.



Scanning Is Referred to as Convolution



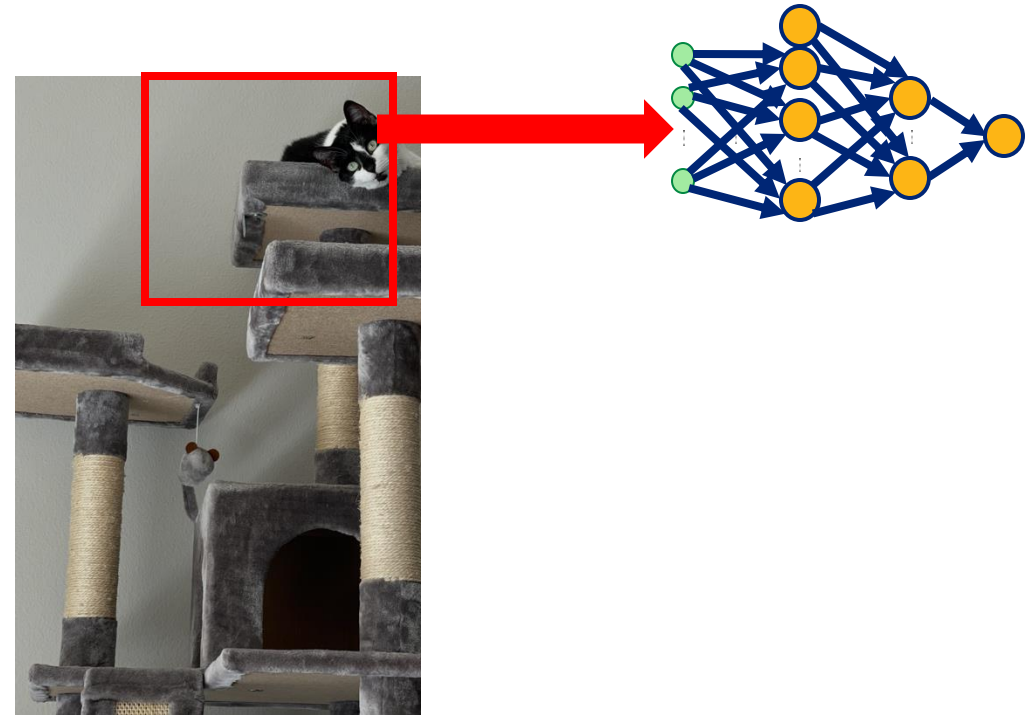
- We need to apply a feature extractor in each region of the image.



Scanning Is Referred to as Convolution



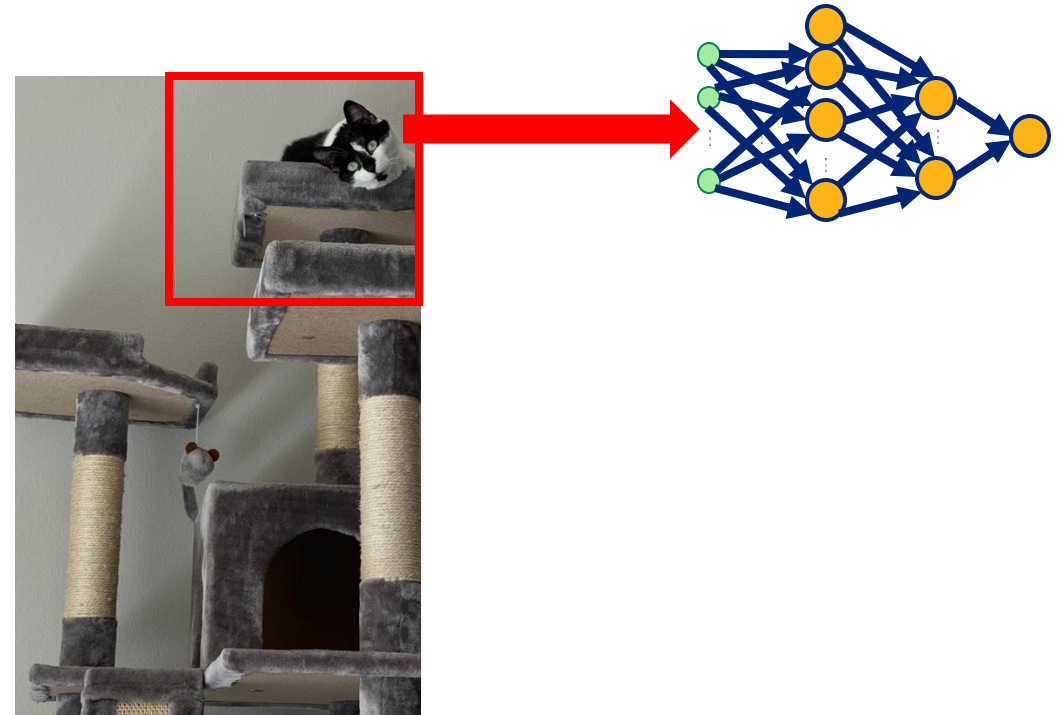
- We need to apply a feature extractor in each region of the image.



Scanning Is Referred to as Convolution



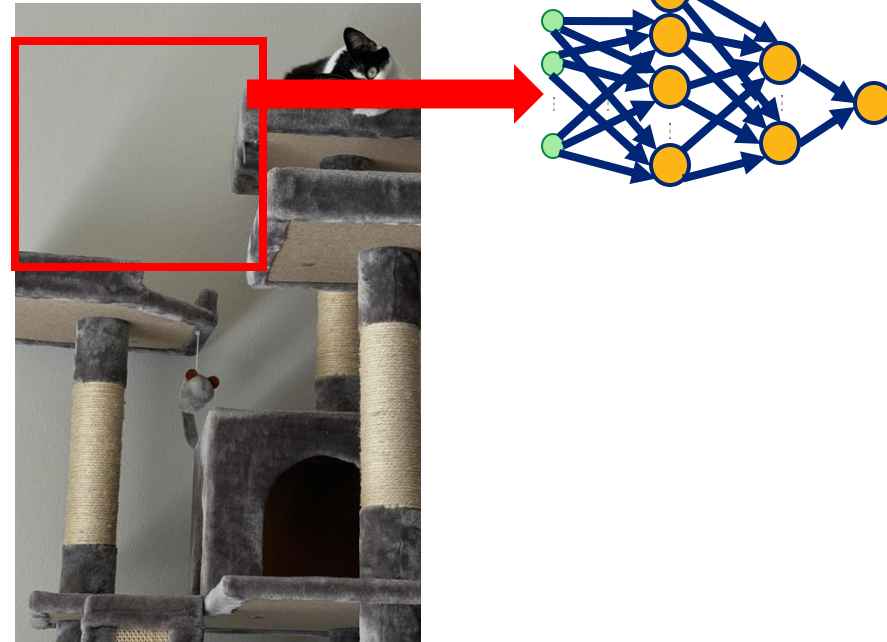
- We need to apply a feature extractor in each region of the image.



Scanning Is Referred to as Convolution



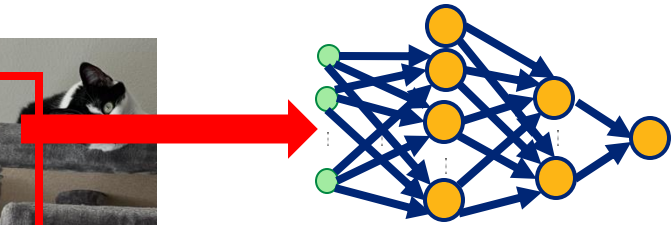
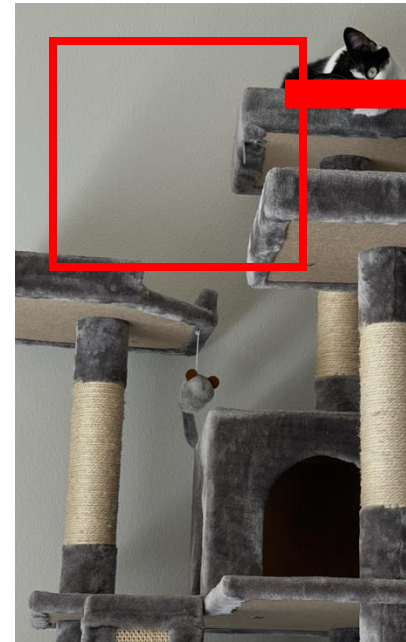
- We need to apply a feature extractor in each region of the image.



Scanning Is Referred to as Convolution



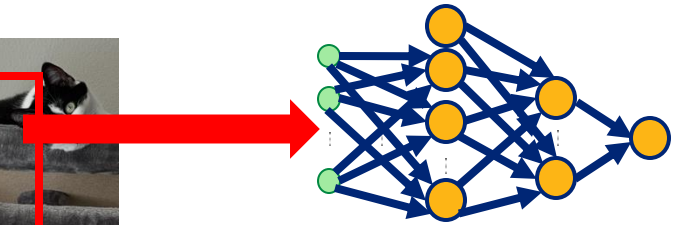
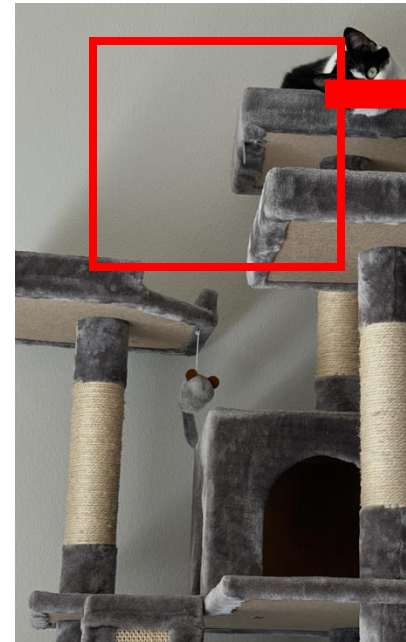
- We need to apply a feature extractor in each region of the image.



Scanning Is Referred to as Convolution



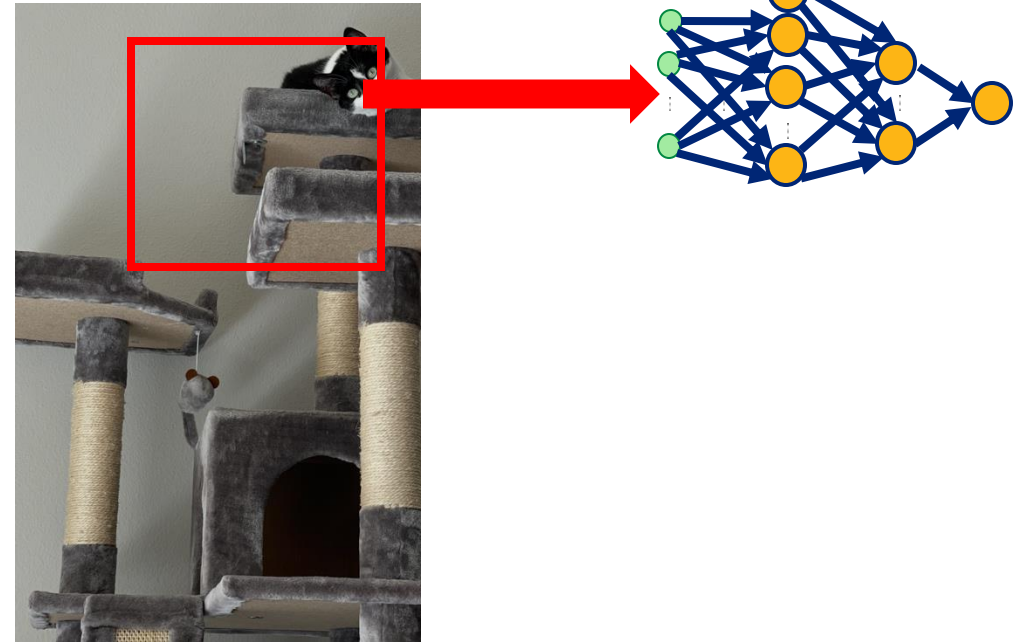
- We need to apply a feature extractor in each region of the image.



Scanning Is Referred to as Convolution



- We need to apply a feature extractor in each region of the image.



Scanning Is Referred to as Convolution

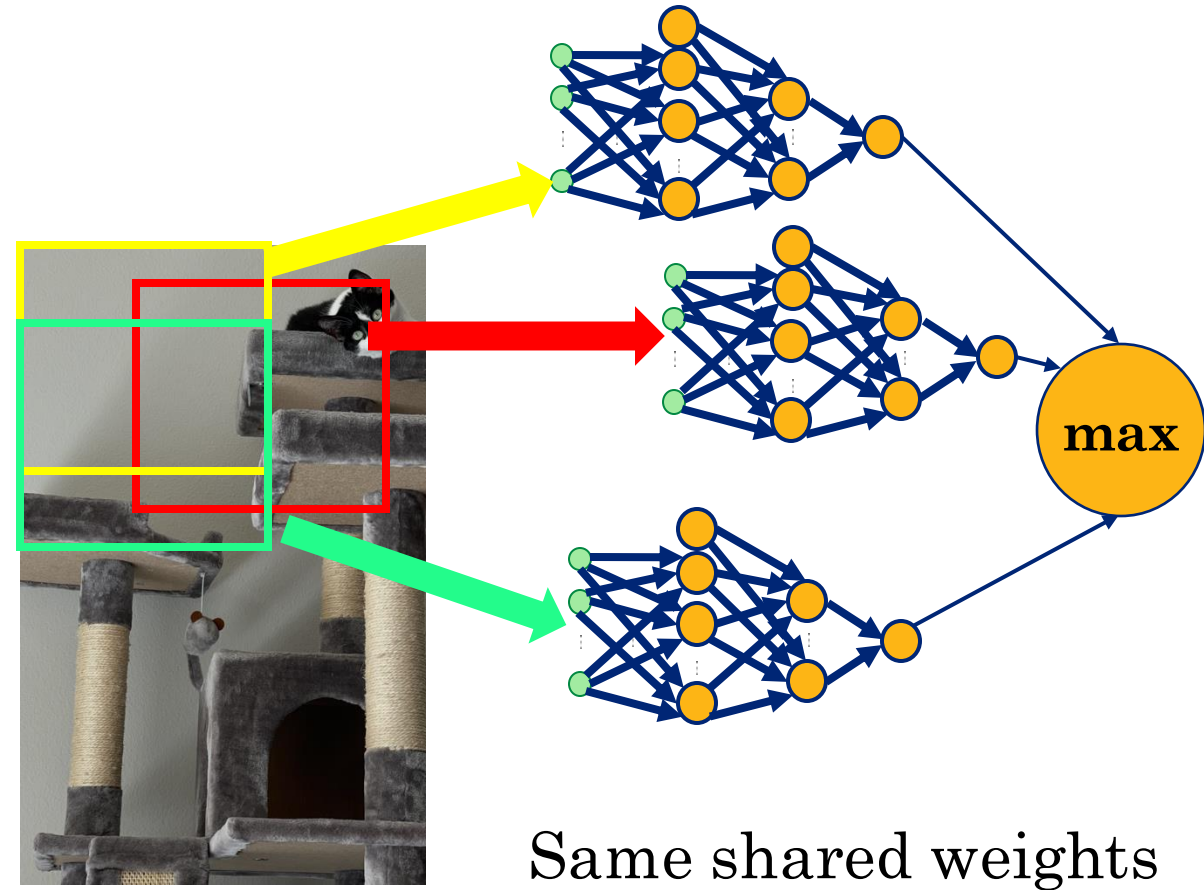


- We need to apply a feature extractor in each region of the image.

Maximum of all the outputs
(Equivalent of Boolean OR)



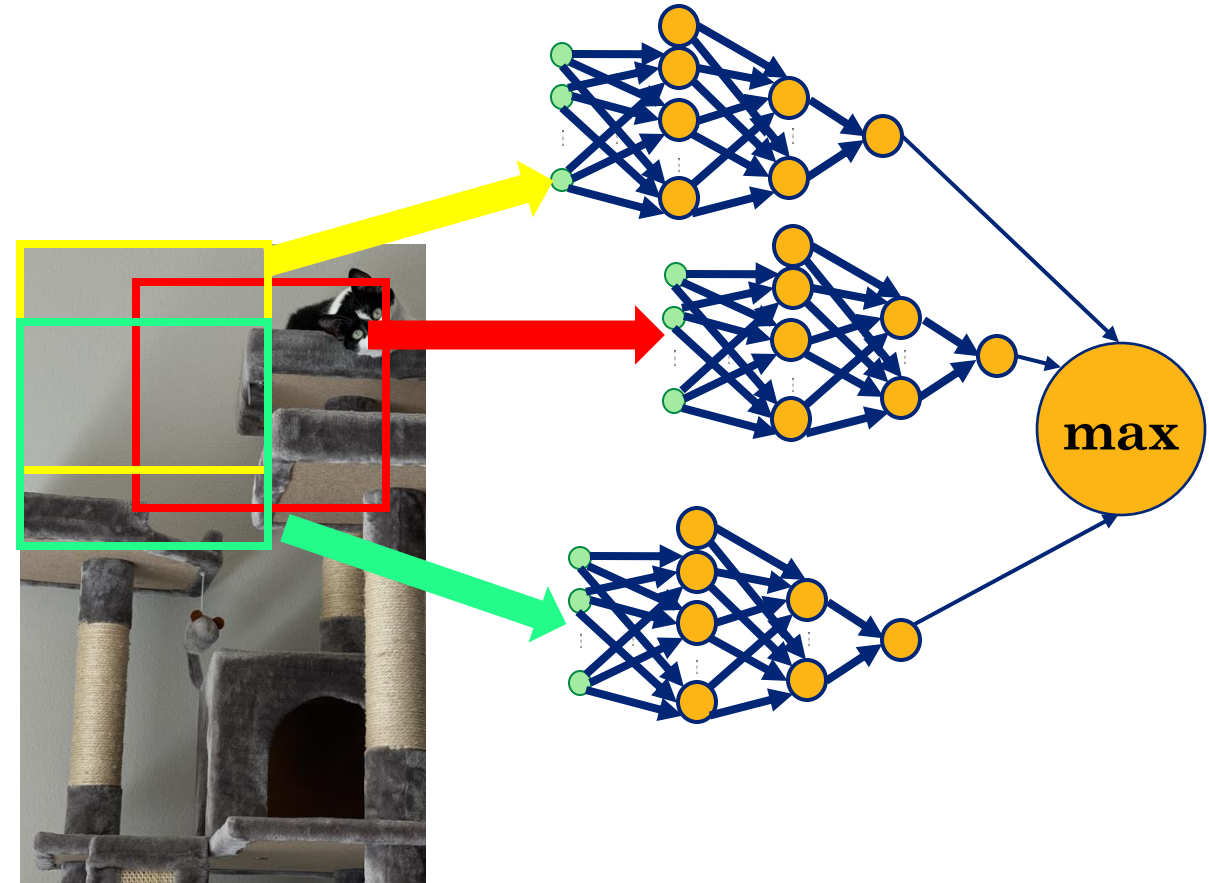
The strength of having the
feature anywhere in the image



Scanning Is Referred to as Convolution



- The entire operation can be viewed as one giant network.
 - With many subnetworks, one per window.
- Restriction: All subnetworks are identical.
- The network is shift-invariant!



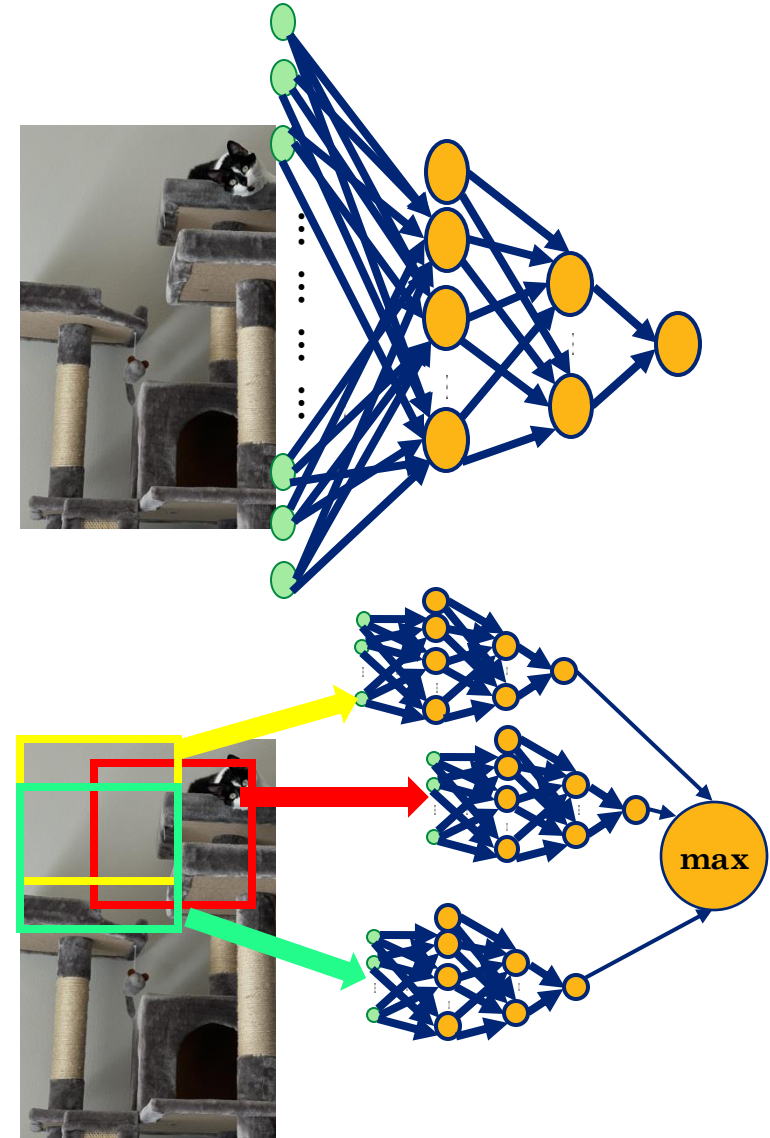


Which of the following is correct?

Regular Network vs. Scanning Network



- In a FC-NN, each neuron is connected by a unique weight to every neuron in the previous layer.
 - Entries in the weight matrix are unique.
- In a scanning NN, each neuron is connected to a subset of neurons in the previous layer
 - The weights matrix is sparse.
 - The weights matrix is block structured with identical blocks.



Element of the Convolutional Neural Network

- Network Design
Considerations for Images
- Element of the
Convolutional Neural
Network
- CNN Design
- CNN History



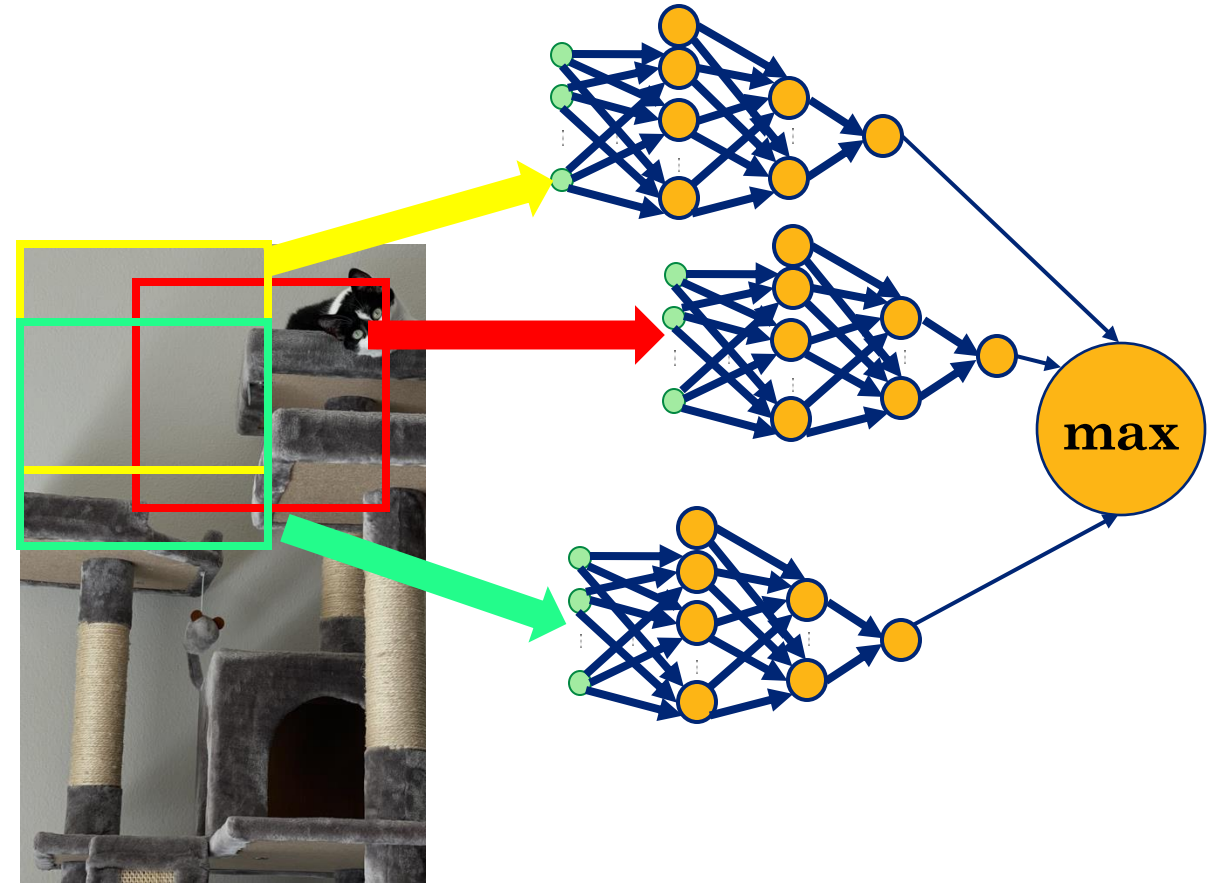
2499499

Back to Scanning Network



- Patches are referred to as "receptive fields".
- The weights form a small two-dimensional grid known as a filter or a kernel.
- Assume we unroll the 2D structures into a vector: output from each receptive field is calculated as:

$$z = \text{ReLU}(w^T x + w_0)$$

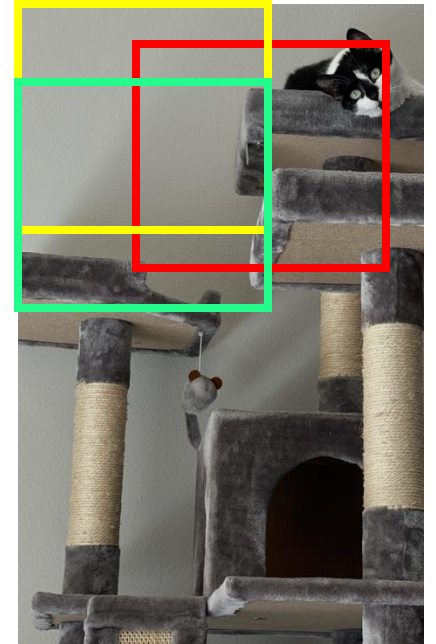


How to Identify a Feature in a Receptive Field?

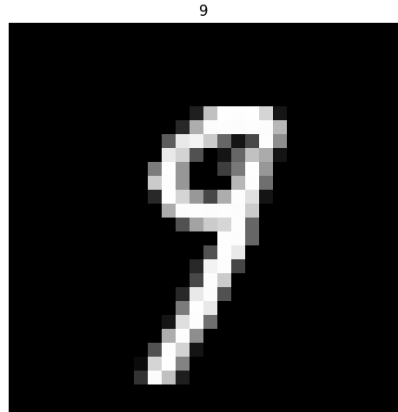


$$z = \text{ReLU}(\mathbf{w}^T \mathbf{x} + w_0)$$

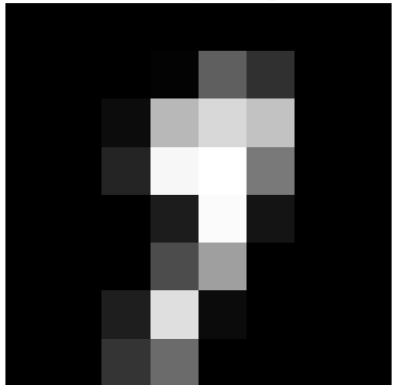
- Suppose that $\mathbf{w}$ and w_0 are fixed and we want to identify $\mathbf{x}$ with the largest output response. Let's also assume that norm $\|\mathbf{x}\|$ is fixed.
- The solution that maximizes $\mathbf{w}^T \mathbf{x}$, and the response of the hidden unit, is of the form $\mathbf{x} = \alpha \mathbf{w}$ for some coefficient α .
- The maximum output response from hidden unit occurs when it detects a patch of image that, up to an overall scaling, looks like the kernel image.



How to Implement Scanning of Features with Weight-Sharing?



Downsampled Image



0	0	0	0	0	0	0	0
0	0	0	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	0	1	1	1	0	0
0	0	0	1	1	0	0	0
0	0	1	1	1	0	0	0
0	0	1	1	0	0	0	0

Feature extractor

Or kernel

-1	-1	-1
2	2	2
-1	-1	-1

Horizontal line
detector

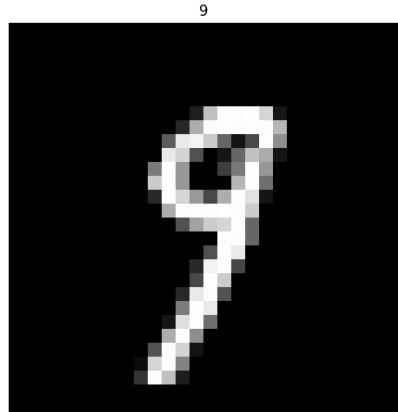
$\odot$

-1

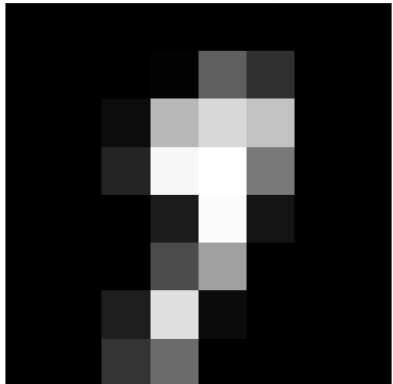
How to Implement Scanning of Features with Weight-Sharing?



2499499



Downsampled Image



Stride = 1

0	0	0	0	0	0	0	0
0	0	0	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	0	1	1	1	0	0
0	0	0	1	1	0	0	0
0	0	1	1	1	0	0	0
0	0	1	1	0	0	0	0

Feature extractor

Or kernel

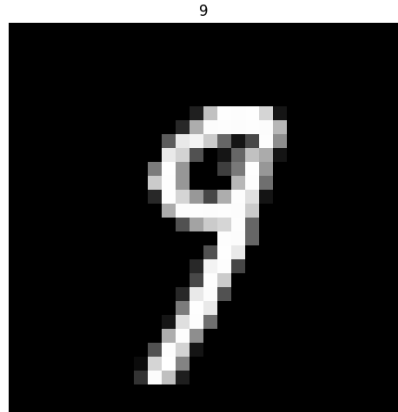
-1	-1	-1
2	2	2
-1	-1	-1

$\odot$

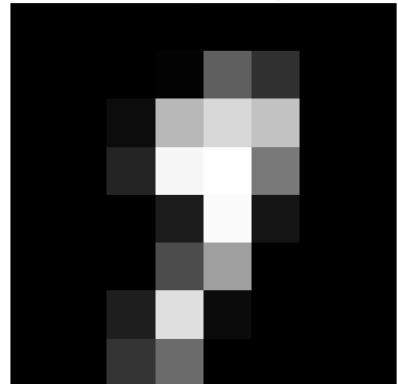
-1 0

Horizontal line
detector

How to Implement Scanning of Features with Weight-Sharing?



Downsampled Image



Stride = 1

0	0	0	0	0	0	0	0
0	0	0	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	0	1	1	1	0	0
0	0	0	1	1	0	0	0
0	0	1	1	1	0	0	0
0	0	1	1	0	0	0	0

Feature extractor

Or kernel

-1	-1	-1
2	2	2
-1	-1	-1

$\odot$

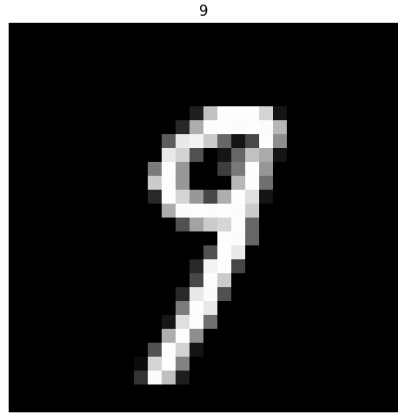
-1 0 1

Horizontal line
detector

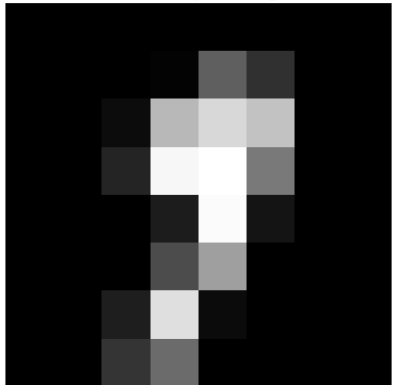
How to Implement Scanning of Features with Weight-Sharing?



2499499



Downsampled Image



Stride = 1

0	0	0	0	0	0	0	0
0	0	0	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	0	1	1	1	0	0
0	0	0	1	1	0	0	0
0	0	1	1	1	0	0	0
0	0	1	1	0	0	0	0

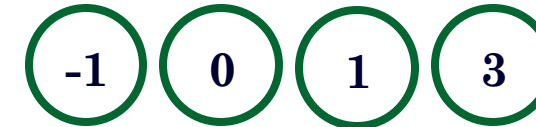
Feature extractor

Or kernel

-1	-1	-1
2	2	2
-1	-1	-1

$\odot$

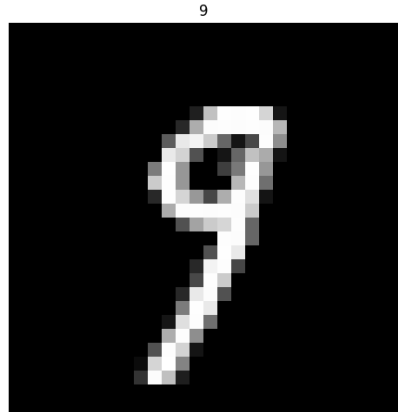
Horizontal line detector



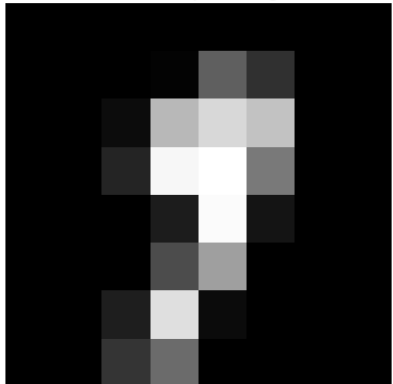
How to Implement Scanning of Features with Weight-Sharing?



2499499



Downsampled Image



Stride = 1

0	0	0	0	0	0	0	0
0	0	0	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	0	1	1	1	0	0
0	0	0	1	1	0	0	0
0	0	1	1	1	0	0	0
0	0	1	1	0	0	0	0

Feature extractor

Or kernel

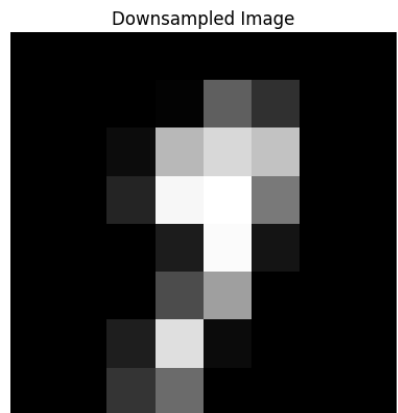
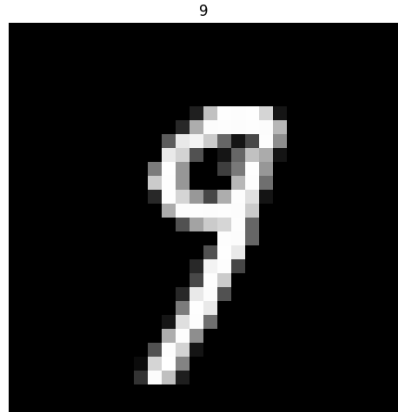
-1	-1	-1
2	2	2
-1	-1	-1

$\odot$

Horizontal line detector



How to Implement Scanning of Features with Weight-Sharing?



Stride = 1

0	0	0	0	0	0	0	0
0	0	0	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	0	1	1	1	0	0
0	0	0	1	1	0	0	0
0	0	1	1	1	0	0	0
0	0	1	1	0	0	0	0

Feature extractor

Or kernel

-1	-1	-1
2	2	2
-1	-1	-1

Horizontal line detector

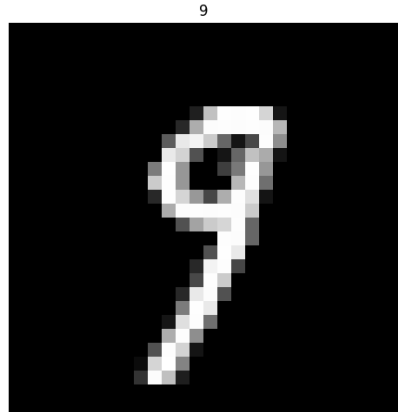
$\odot$



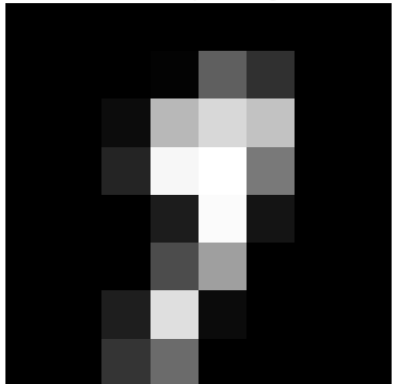
How to Implement Scanning of Features with Weight-Sharing?



2499499



Downsampled Image



Stride = 1

0	0	0	0	0	0	0	0
0	0	0	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	0	1	1	1	0	0
0	0	0	1	1	0	0	0
0	0	1	1	1	0	0	0
0	0	1	1	0	0	0	0

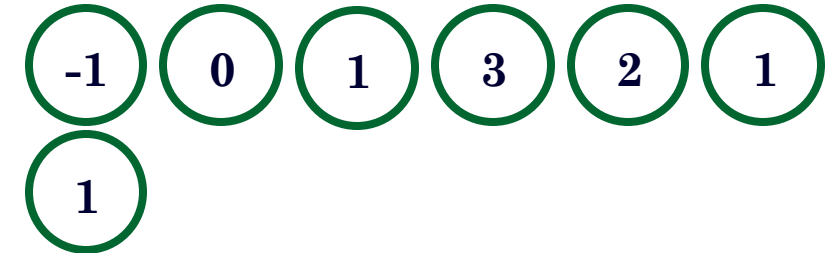
Feature extractor

Or kernel

-1	-1	-1
2	2	2
-1	-1	-1

$\odot$

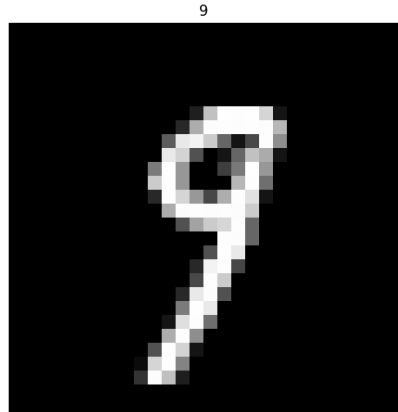
Horizontal line detector



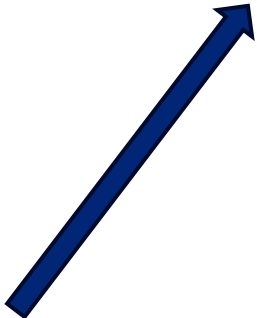
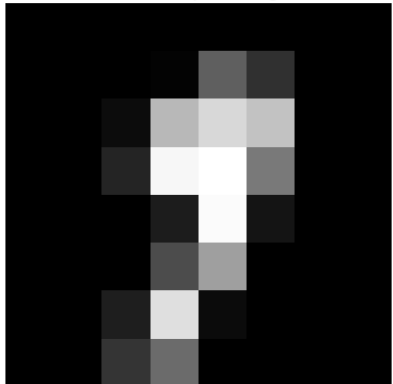
How to Implement Scanning of Features with Weight-Sharing?



2499499



Downsampled Image



Stride = 1

0	0	0	0	0	0	0	0
0	0	0	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	0	1	1	1	0	0
0	0	0	1	1	0	0	0
0	0	1	1	1	0	0	0
0	0	1	1	0	0	0	0

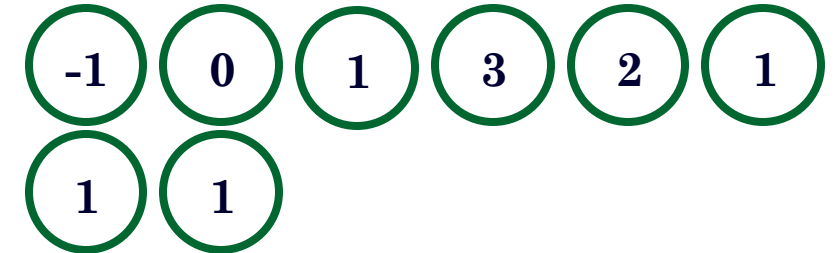
Feature extractor

Or kernel

-1	-1	-1
2	2	2
-1	-1	-1

$\odot$

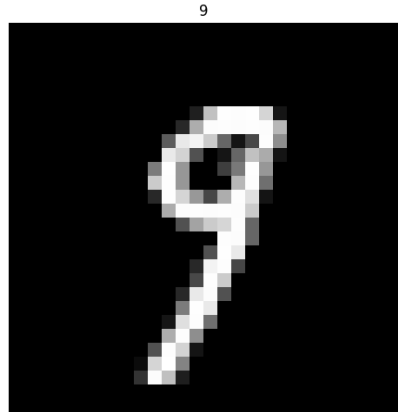
Horizontal line detector



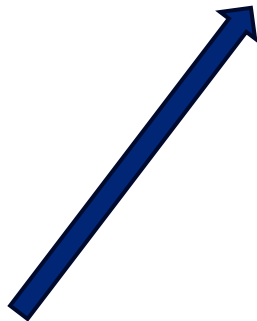
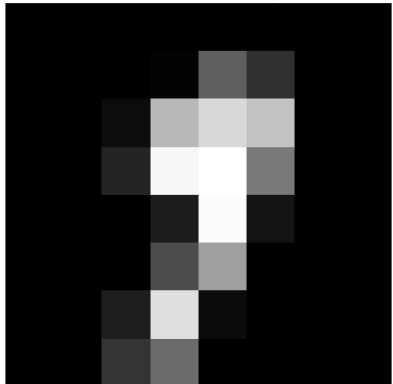
How to Implement Scanning of Features with Weight-Sharing?



2499499



Downsampled Image



Stride = 1

0	0	0	0	0	0	0	0
0	0	0	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	0	1	1	1	0	0
0	0	0	1	1	0	0	0
0	0	1	1	1	0	0	0
0	0	1	1	0	0	0	0

Feature extractor

Or kernel

-1	-1	-1
2	2	2
-1	-1	-1

$\odot$

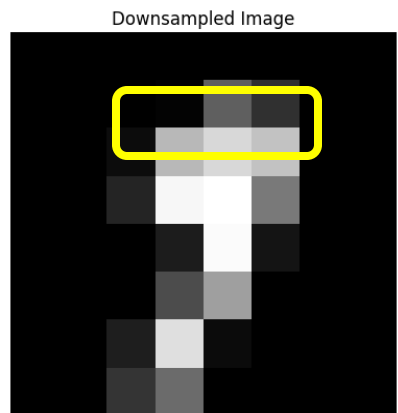
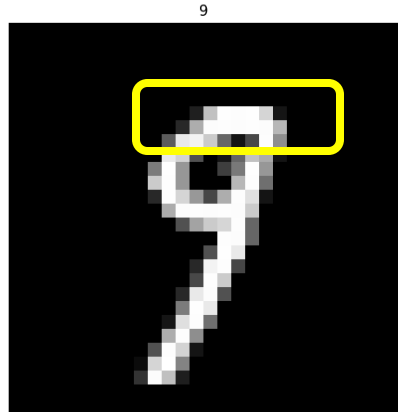
Horizontal line detector

-1	0	1	3	2	1
1	1	1	0	0	0
1	1	1	0	0	0
-1	-1	-1	1	1	1
-1	-1	-1	-1	-1	-1
1	1	2	1	1	0

How to Implement Scanning of Features with Weight-Sharing?



2499499



Stride = 1

0	0	0	0	0	0	0	0
0	0	0	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	0	1	1	1	0	0
0	0	0	1	1	0	0	0
0	0	1	1	1	0	0	0
0	0	1	1	0	0	0	0

Feature extractor

Or kernel

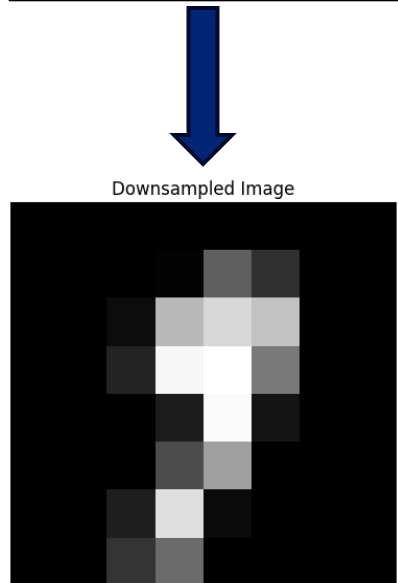
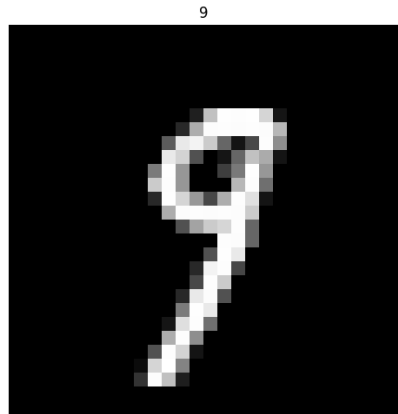
-1	-1	-1
2	2	2
-1	-1	-1

$\odot$

Horizontal line detector

-1	0	1	3	2	1
1	1	1	0	0	0
1	1	1	0	0	0
-1	-1	-1	1	1	1
-1	-1	-1	-1	-1	-1
1	1	2	1	1	0

How to Implement Scanning of Features with Weight-Sharing?



Stride = 1

0	0	0	0	0	0	0	0
0	0	0	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	0	1	1	1	0	0
0	0	0	1	1	0	0	0
0	0	1	1	1	0	0	0
0	0	1	1	0	0	0	0

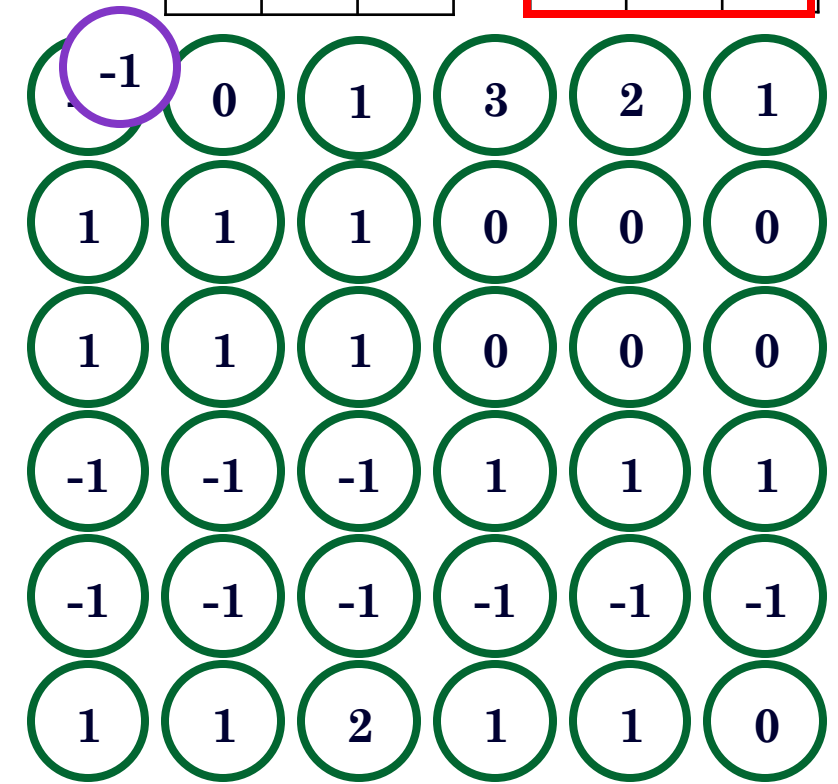
Horizontal line
detector

-1	-1	-1
2	2	2
-1	-1	-1

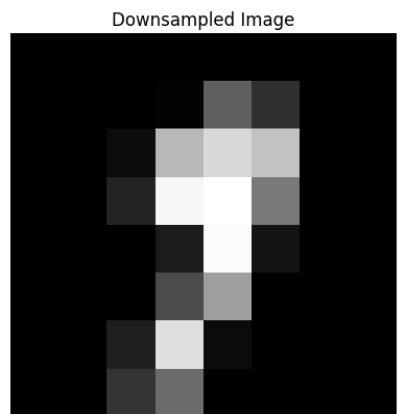
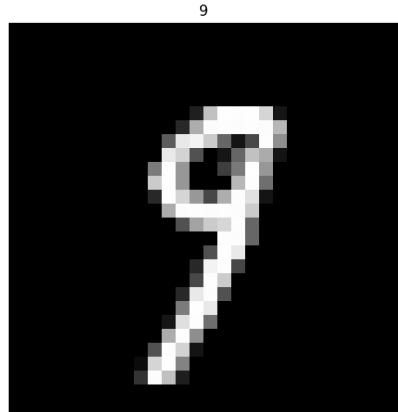
Vertical line
detector

-1	2	-1
-1	2	-1
-1	2	-1

$\odot$



How to Implement Scanning of Features with Weight-Sharing?



Stride = 1

0	0	0	0	0	0	0	0
0	0	0	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	0	1	1	1	0	0
0	0	0	1	1	0	0	0
0	0	1	1	1	0	0	0
0	0	1	1	0	0	0	0

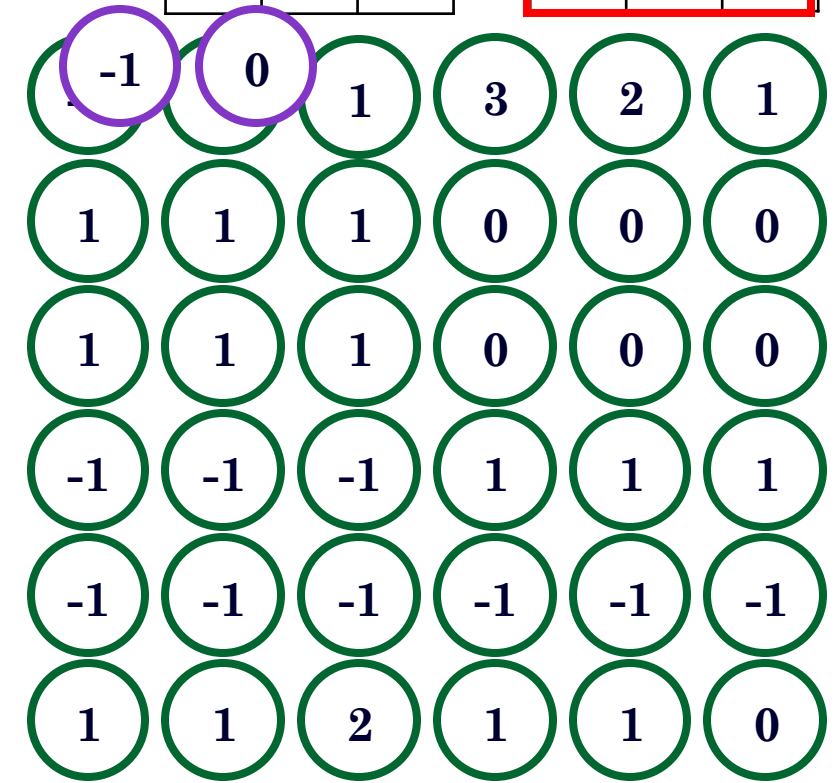
Horizontal line
detector

-1	-1	-1
2	2	2
-1	-1	-1

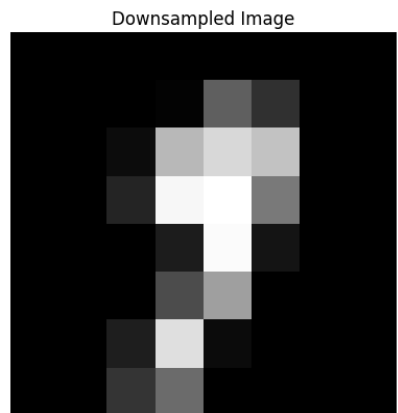
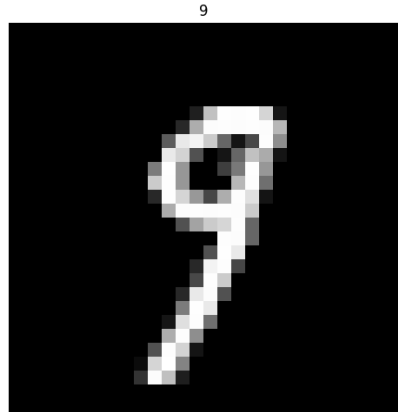
Vertical line
detector

-1	2	-1
-1	2	-1
-1	2	-1

$\odot$



How to Implement Scanning of Features with Weight-Sharing?



Stride = 1

0	0	0	0	0	0	0	0
0	0	0	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	0	1	1	1	0	0
0	0	0	1	1	0	0	0
0	0	1	1	1	0	0	0
0	0	1	1	0	0	0	0

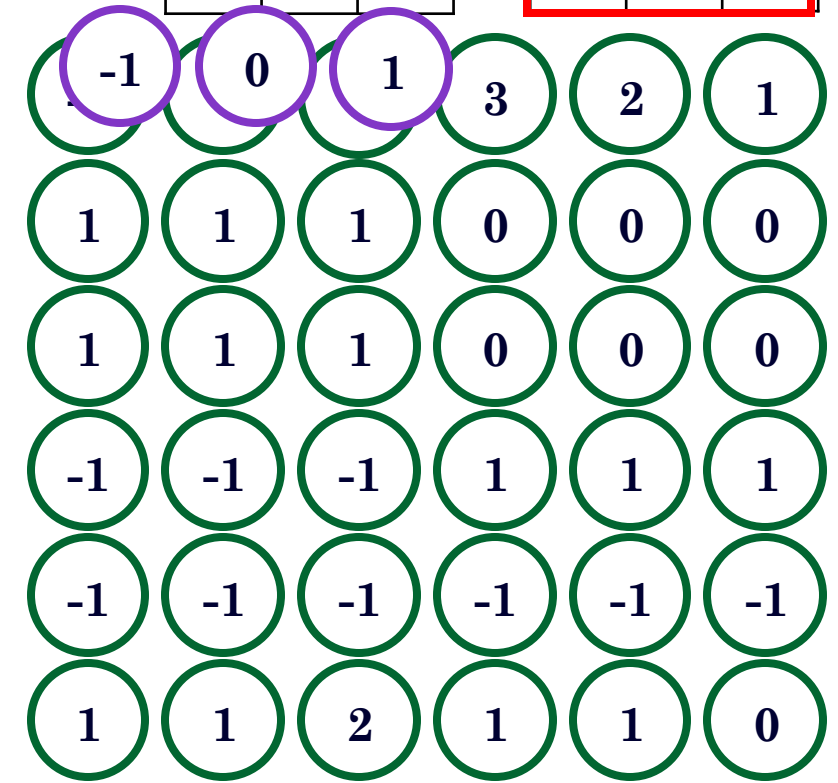
Horizontal line
detector

-1	-1	-1
2	2	2
-1	-1	-1

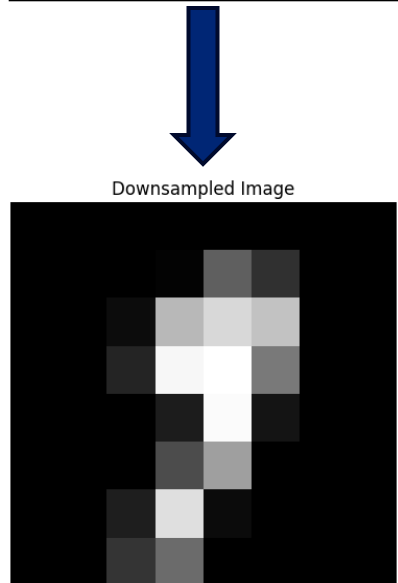
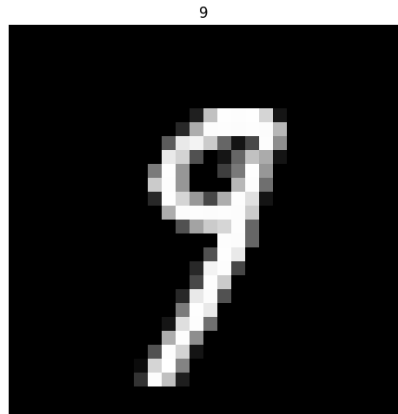
Vertical line
detector

-1	2	-1
-1	2	-1
-1	2	-1

$\odot$



How to Implement Scanning of Features with Weight-Sharing?



Stride = 1

0	0	0	0	0	0	0	0
0	0	0	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	0	1	1	1	0	0
0	0	0	1	1	0	0	0
0	0	1	1	1	0	0	0
0	0	1	1	0	0	0	0

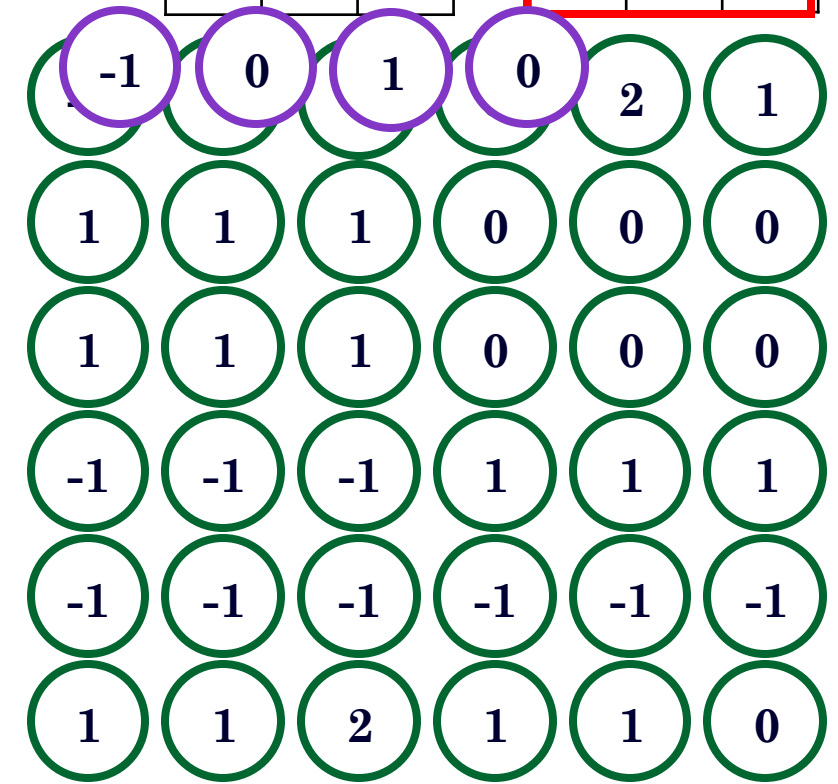
Horizontal line
detector

-1	-1	-1
2	2	2
-1	-1	-1

Vertical line
detector

-1	2	-1
-1	2	-1
-1	2	-1

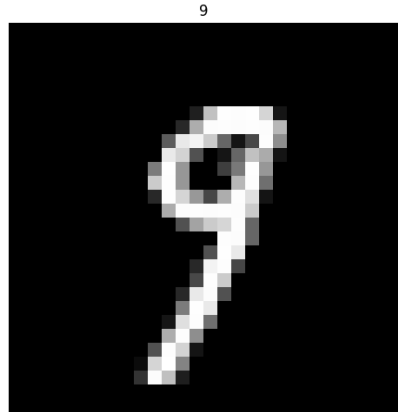
$\odot$



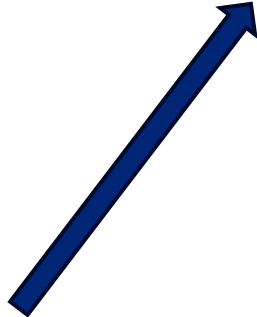
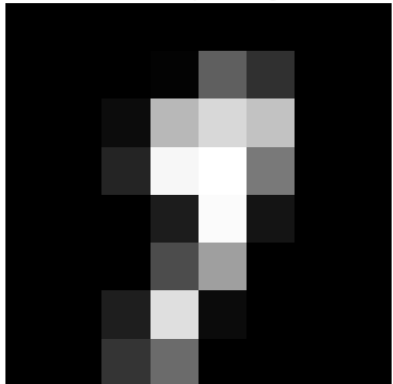
How to Implement Scanning of Features with Weight-Sharing?



2499499



Downsampled Image



Stride = 1

0	0	0	0	0	0	0	0
0	0	0	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	0	1	1	1	0	0
0	0	0	1	1	0	0	0
0	0	1	1	1	0	0	0
0	0	1	1	0	0	0	0

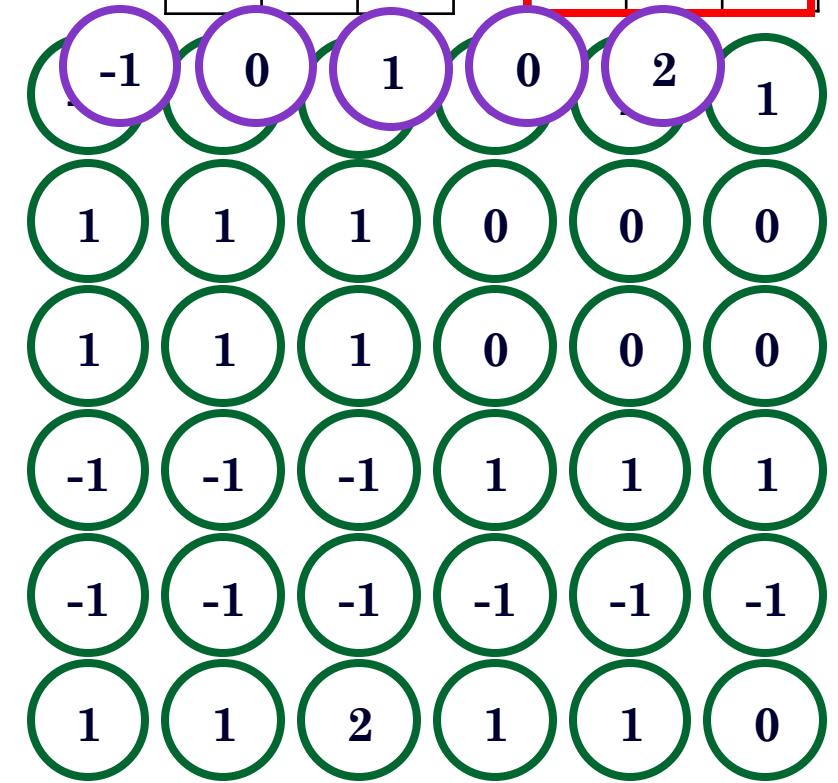
Horizontal line
detector

-1	-1	-1
2	2	2
-1	-1	-1

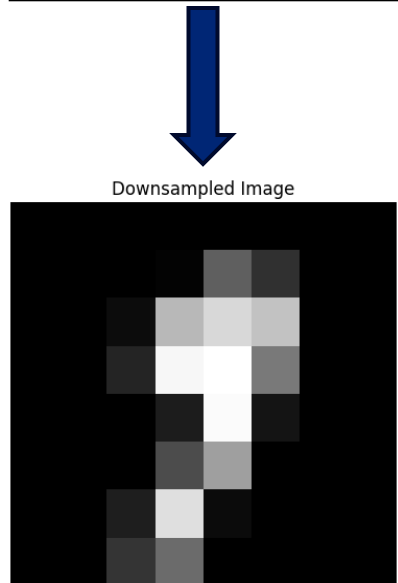
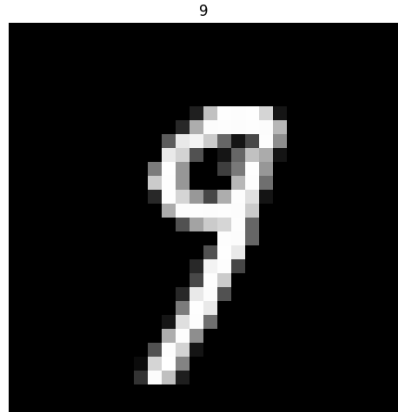
Vertical line
detector

-1	2	-1
-1	2	-1
-1	2	-1

$\odot$



How to Implement Scanning of Features with Weight-Sharing?



Stride = 1

0	0	0	0	0	0	0	0
0	0	0	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	0	1	1	1	0	0
0	0	0	1	1	0	0	0
0	0	1	1	1	0	0	0
0	0	1	1	0	0	0	0

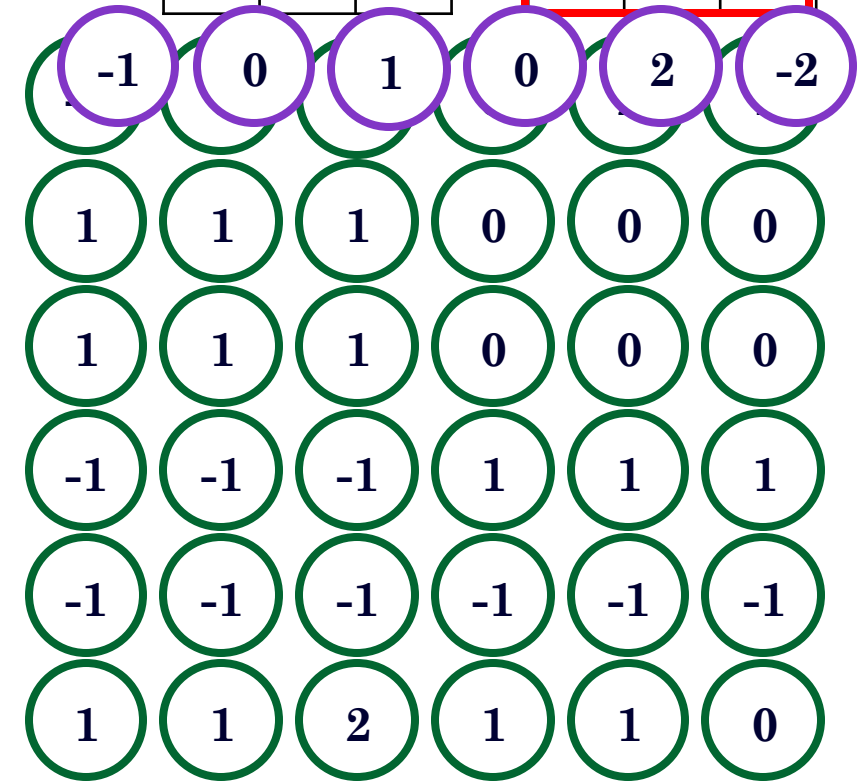
Horizontal line
detector

-1	-1	-1
2	2	2
-1	-1	-1

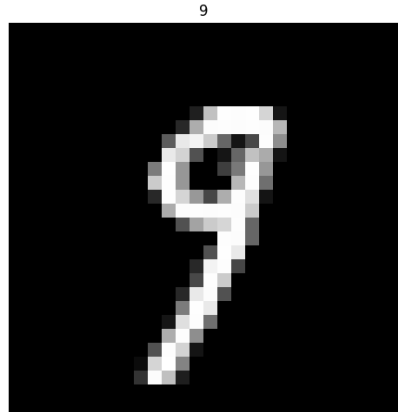
Vertical line
detector

-1	2	-1
-1	2	-1
-1	2	-1

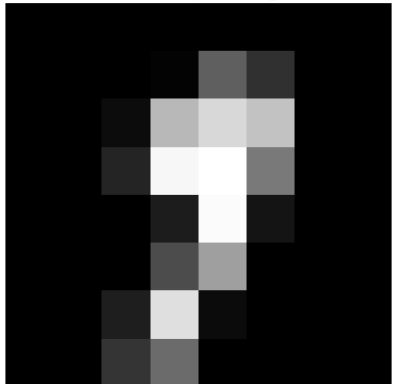
$\odot$



How to Implement Scanning of Features with Weight-Sharing?



Downsampled Image



Stride = 1

0	0	0	0	0	0	0	0
0	0	0	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	0	1	1	1	0	0
0	0	0	1	1	0	0	0
0	0	1	1	1	0	0	0
0	0	1	1	0	0	0	0

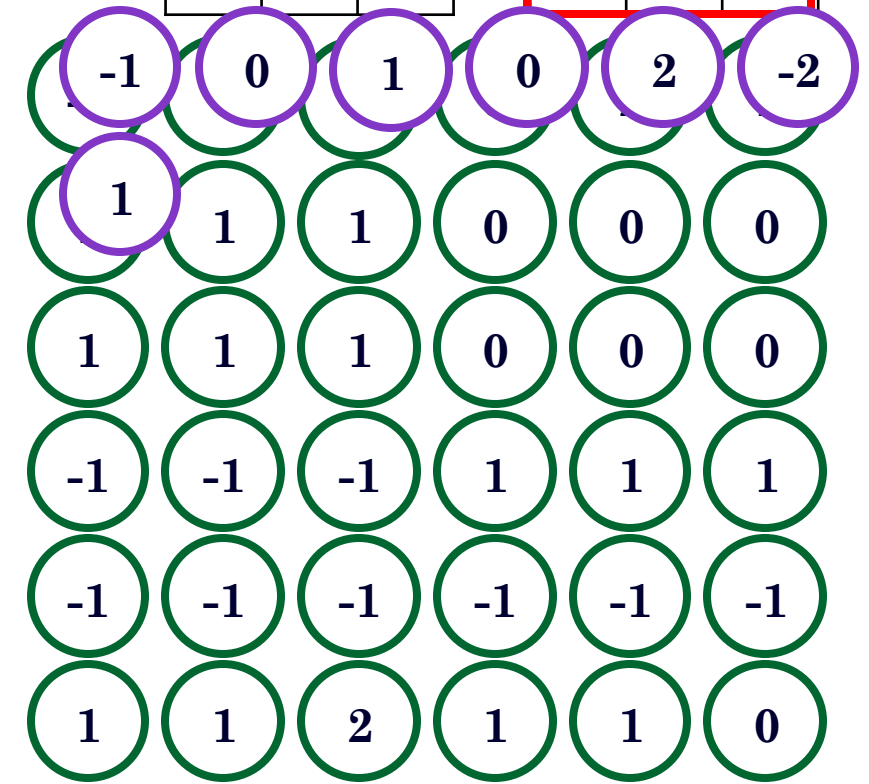
Horizontal line
detector

-1	-1	-1
2	2	2
-1	-1	-1

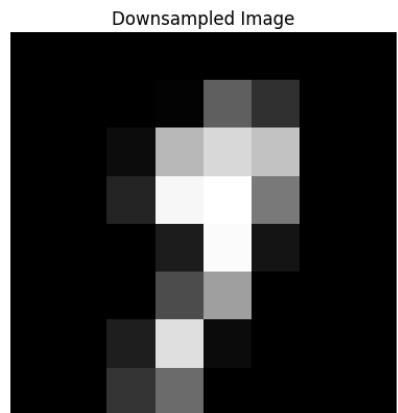
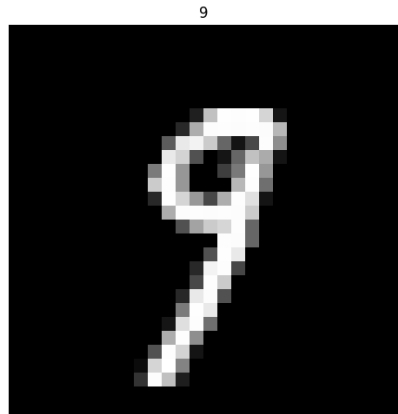
Vertical line
detector

-1	2	-1
-1	2	-1
-1	2	-1

$\odot$



How to Implement Scanning of Features with Weight-Sharing?



Stride = 1

0	0	0	0	0	0	0	0
0	0	0	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	1	1	1	1	0	0
0	0	0	1	1	1	0	0
0	0	0	1	1	0	0	0
0	0	1	1	1	0	0	0
0	0	1	1	0	0	0	0

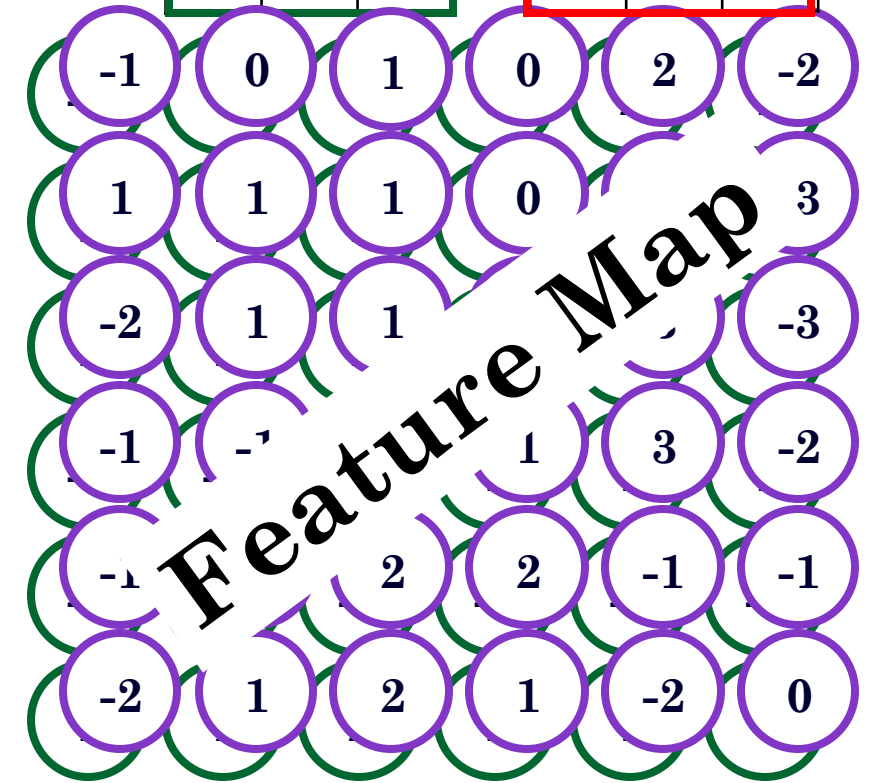
Horizontal line
detector

-1	-1	-1
2	2	2
-1	-1	-1

Vertical line
detector

-1	2	-1
-1	2	-1
-1	2	-1

$\odot$

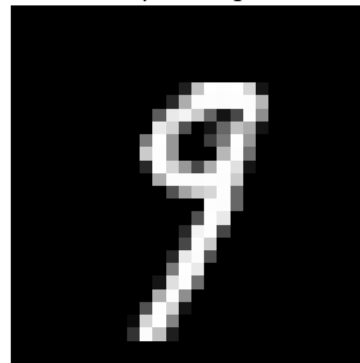


Convolution Example

Kernel: horizontal line



Input Image



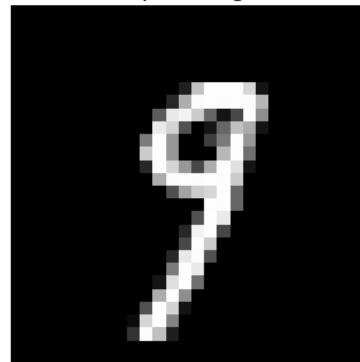
Convolved Output



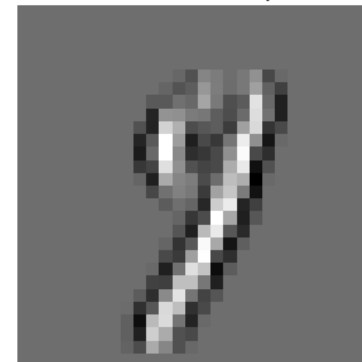
Kernel: vertical line



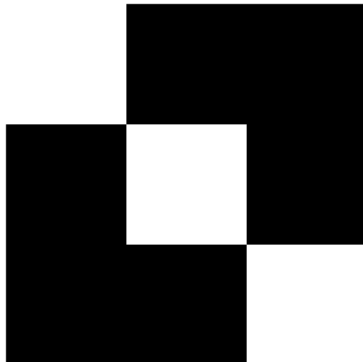
Input Image



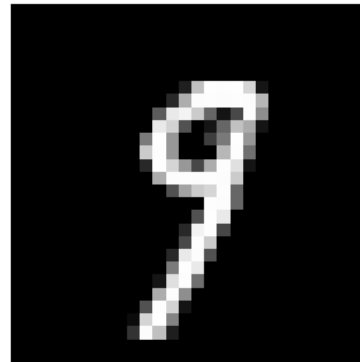
Convolved Output



Kernel: diagonal line



Input Image



Convolved Output



2499499



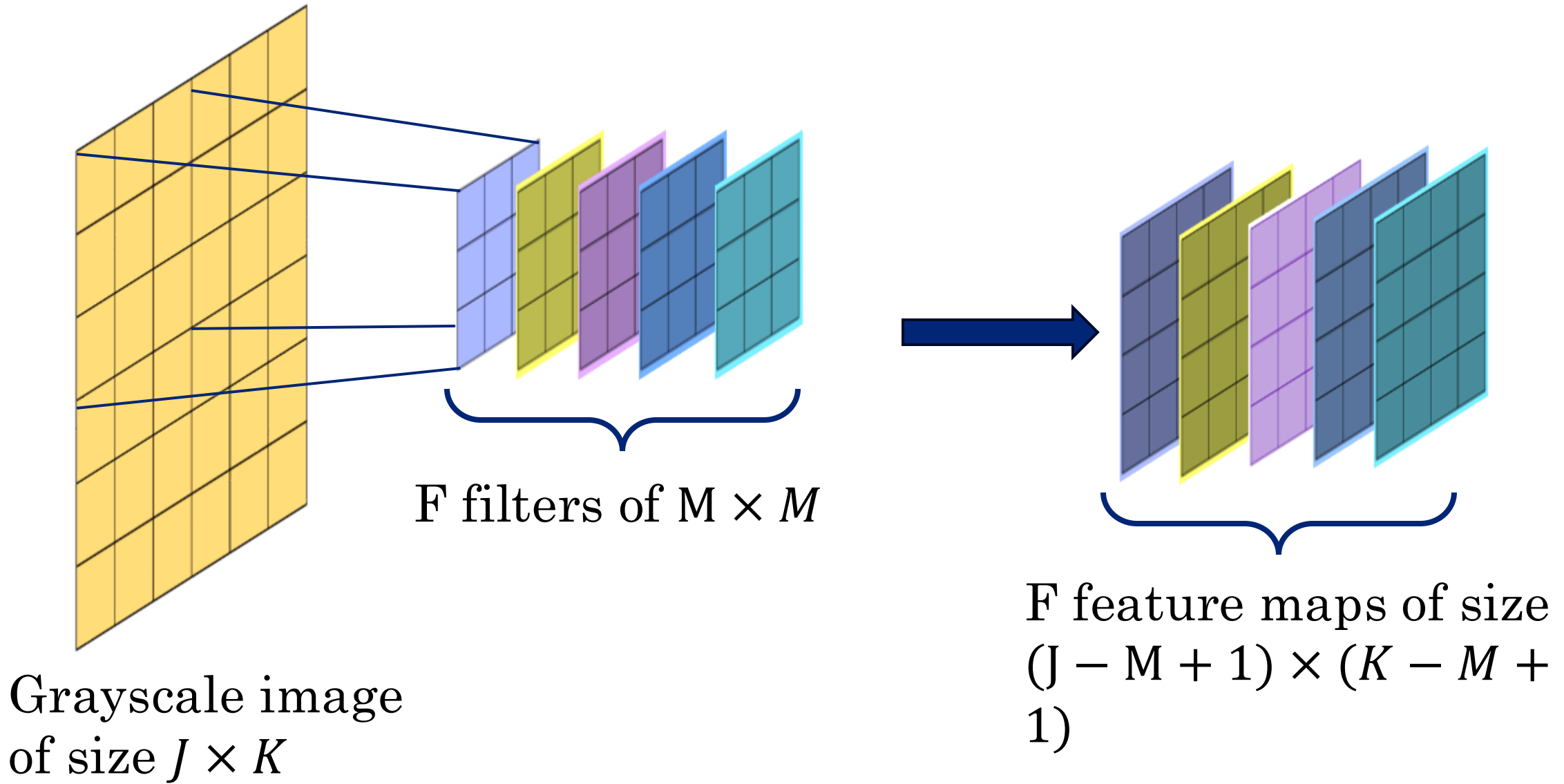
If we convolve a kernel of size 5×5 with an image of size 8×8 with the stride of 1, what are the output dimensions?



If we convolve 4 kernels of size 5×5 with an image of size 8×8 with the stride of 1, what are the output dimensions?

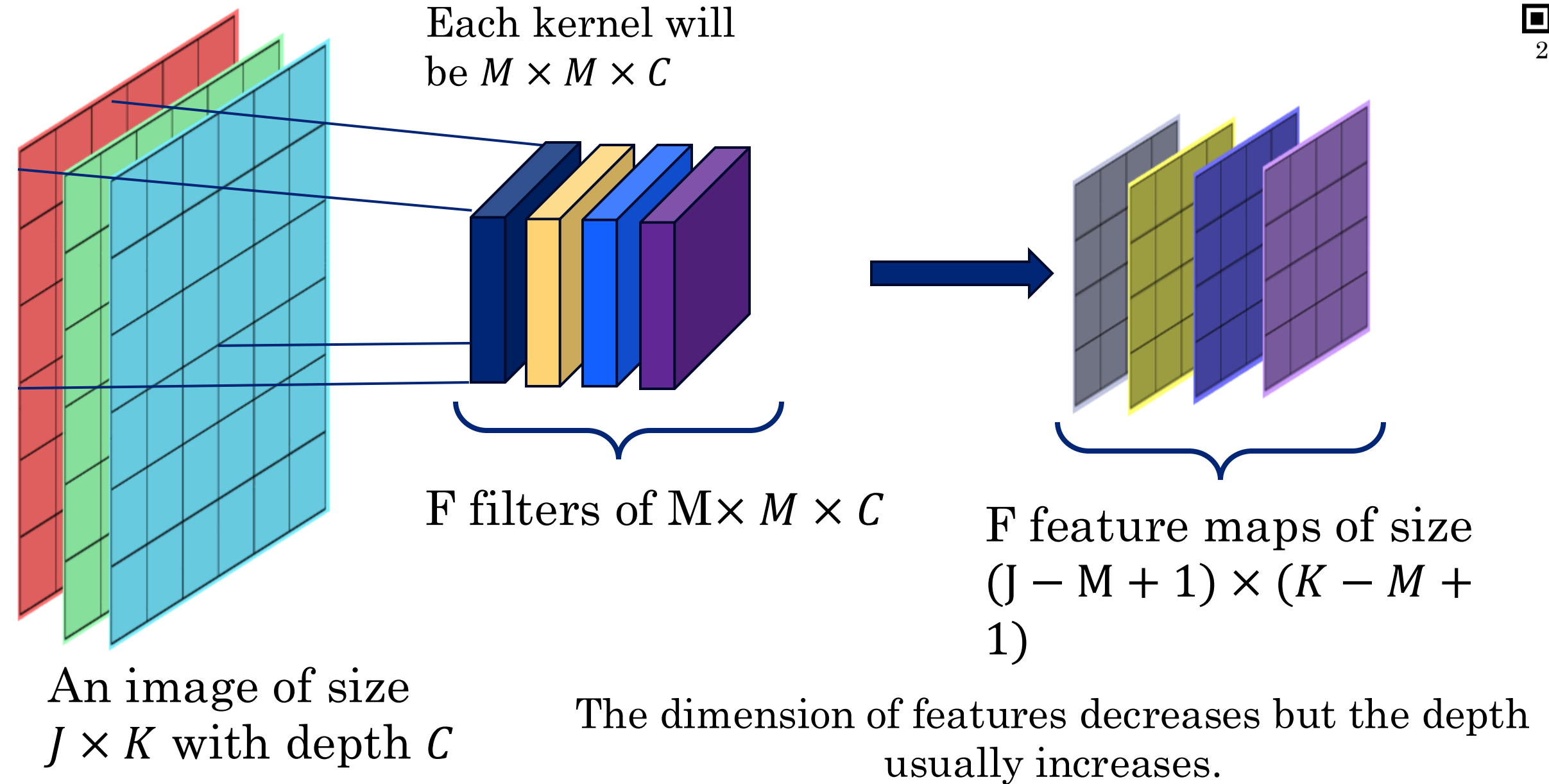


2499499





2499499



What If We Want to Keep the Same Image Dimensions?

- We can use padding.

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	1	1	1	0	0	0
0	0	0	1	1	1	1	0	0	0
0	0	0	1	1	1	1	0	0	0
0	0	0	0	1	1	1	0	0	0
0	0	0	0	1	1	0	0	0	0
0	0	0	1	1	1	0	0	0	0
0	0	0	1	1	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

Feature extractor

Or kernel

-1	-1	-1
2	2	2
-1	-1	-1

Horizontal line
detector

$\odot$

0



2499499

What If We Want to Keep the Same Image Dimensions?



- We can use padding.

If we pad with P rows and columns on each side, the output dimensions of a

feature map will be:

$$(J + 2P - M + 1) \times (K + 2P - M + 1)$$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	1	1	1	0	0	0
0	0	0	1	1	1	1	0	0	0
0	0	0	1	1	1	1	0	0	0
0	0	0	0	1	1	1	0	0	0
0	0	0	0	1	1	0	0	0	0
0	0	0	1	1	1	0	0	0	0
0	0	0	1	1	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

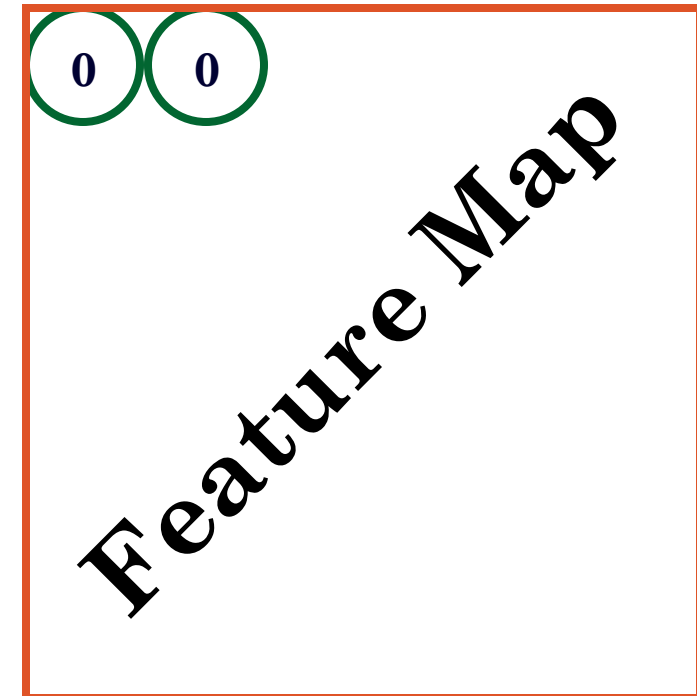
Feature extractor

Or kernel

-1	-1	-1
2	2	2
-1	-1	-1

$\odot$

Horizontal line detector



Avoiding Redundancy



- We do not need to move one pixel at a time.

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	1	1	1	0	0	0
0	0	0	1	1	1	1	0	0	0
0	0	0	1	1	1	1	0	0	0
0	0	0	0	1	1	1	0	0	0
0	0	0	0	1	1	0	0	0	0
0	0	0	1	1	1	0	0	0	0
0	0	0	1	1	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

Feature extractor

Or kernel

-1	-1	-1
2	2	2
-1	-1	-1

Horizontal line
detector

$\odot$

0

Stride = 2

Avoiding Redundancy



- We do not need to move one pixel at a time.

If we pad with P rows and columns on each side, and move with the stride of S, the output dimensions of a feature map will be:

$$\left(\left\lfloor \frac{J + 2P - M}{S} \right\rfloor + 1 \right) \times \left(\left\lfloor \frac{J + 2P - M}{S} \right\rfloor + 1 \right)$$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	1	1	1	0	0	0
0	0	0	1	1	1	1	0	0	0
0	0	0	1	1	1	1	0	0	0
0	0	0	0	1	1	1	0	0	0
0	0	0	0	1	1	0	0	0	0
0	0	0	1	1	1	0	0	0	0
0	0	0	1	1	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

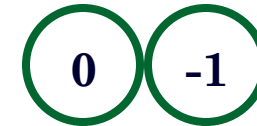
Feature extractor

Or kernel

-1	-1	-1
2	2	2
-1	-1	-1

Horizontal line detector

⊙

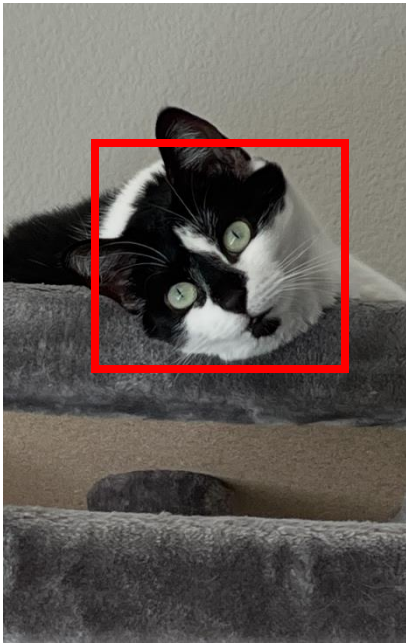


Stride = 2

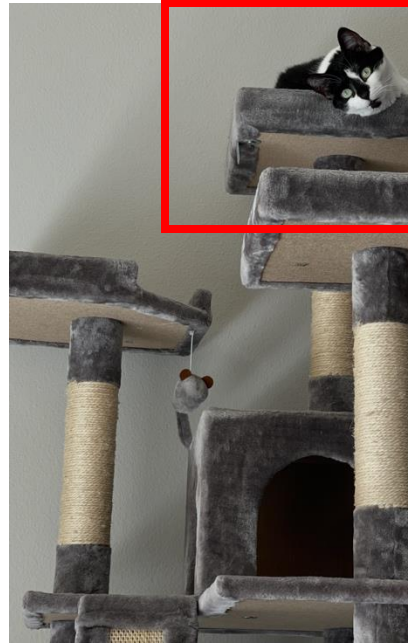
Characteristics of Image Analysis



Patterns are usually smaller than high-resolution images.



Patterns can appear anywhere in the image.

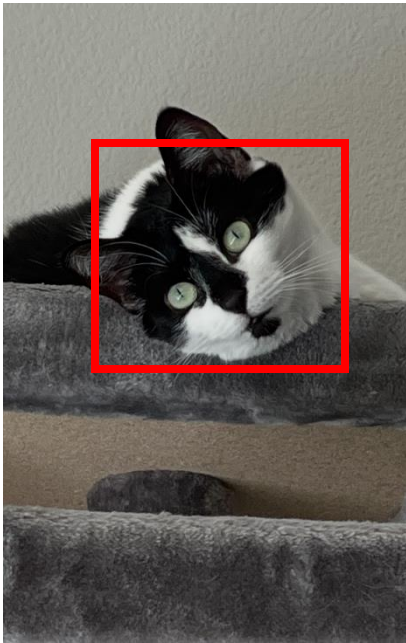


Addressed with
convolution

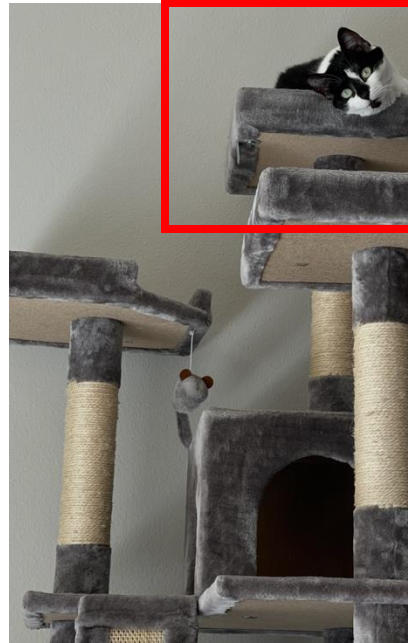
Characteristics of Image Analysis



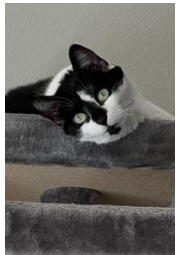
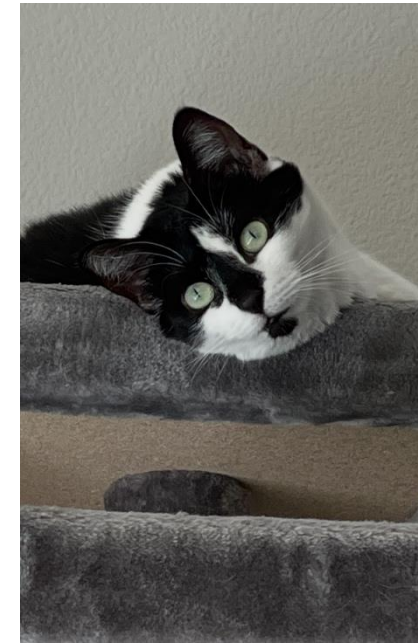
Patterns are usually smaller than high-resolution images.



Patterns can appear anywhere in the image.



Patterns might appear at different scales.



How Can We Detect Features at Different Scale?



2499499

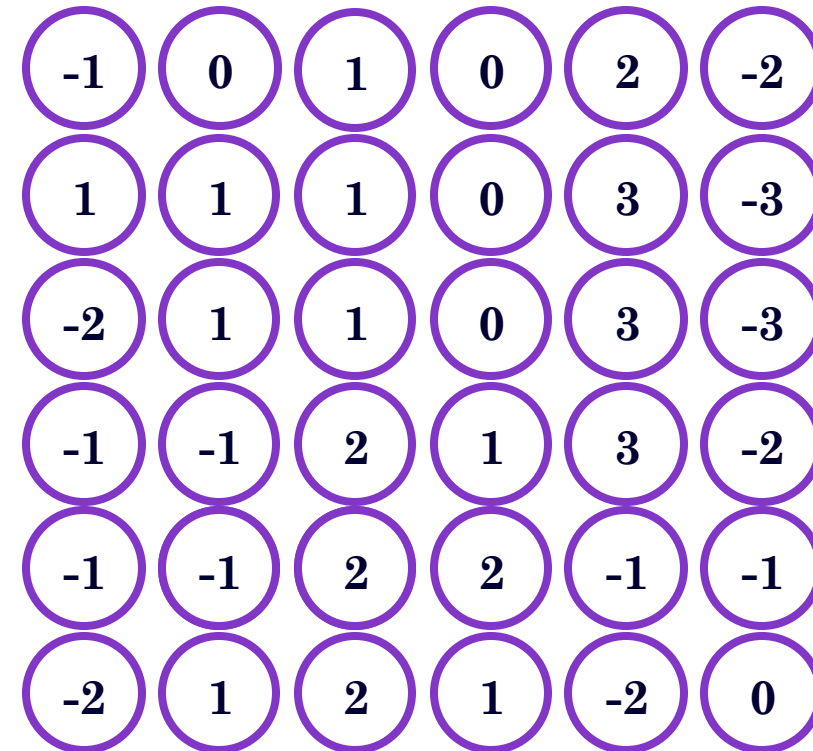
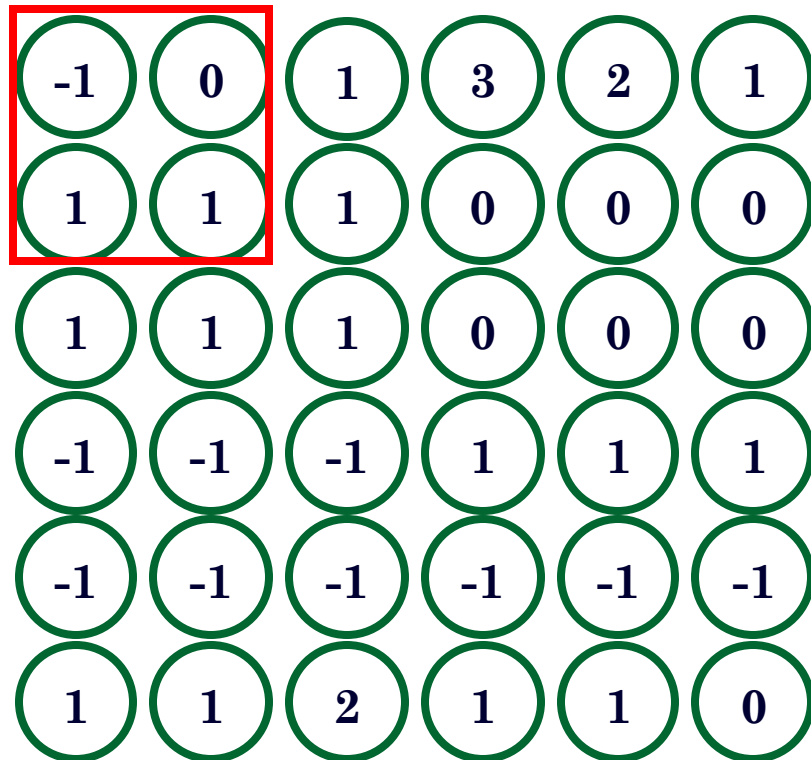
2×2 pooling

-1	-1	-1
2	2	2
-1	-1	-1

Down-sampling features

This is called pooling.

-1	2	-1
-1	2	-1
-1	2	-1



How Can We Detect Features at Different Scale?



2499499

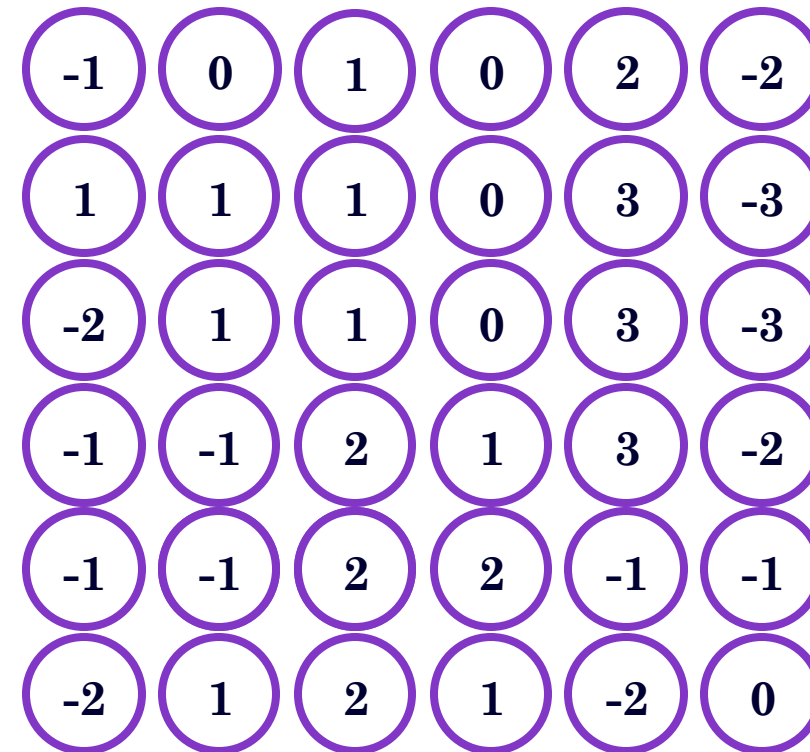
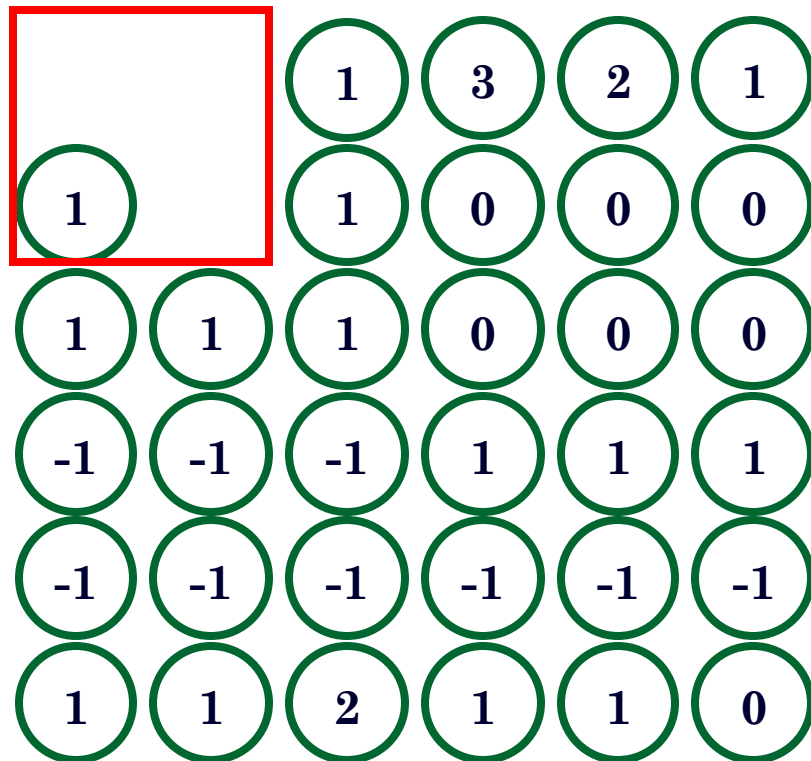
2×2 pooling

-1	-1	-1
2	2	2
-1	-1	-1

Down-sampling features

This is called pooling.

-1	2	-1
-1	2	-1
-1	2	-1



How Can We Detect Features at Different Scale?



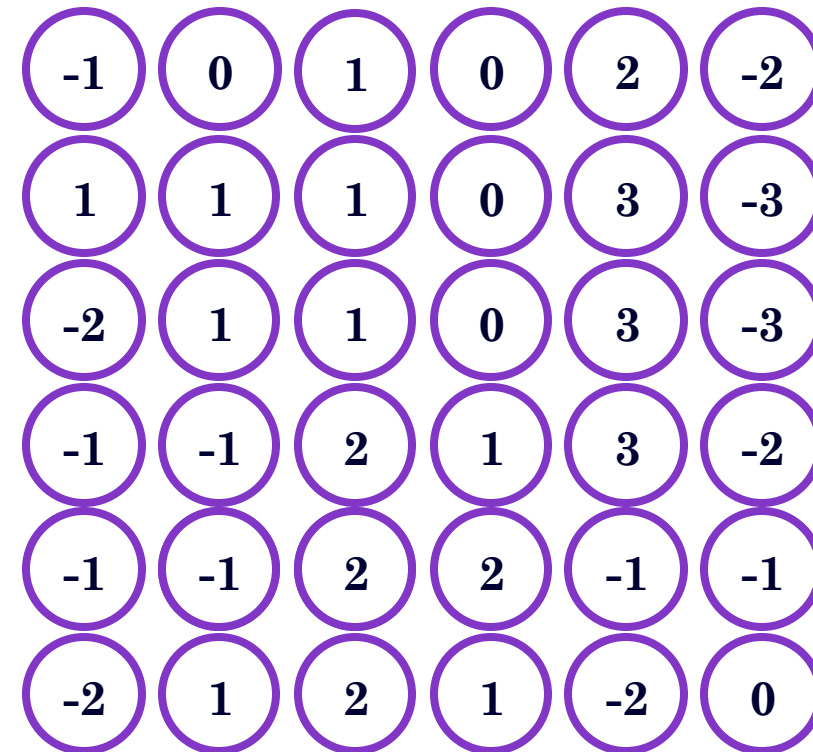
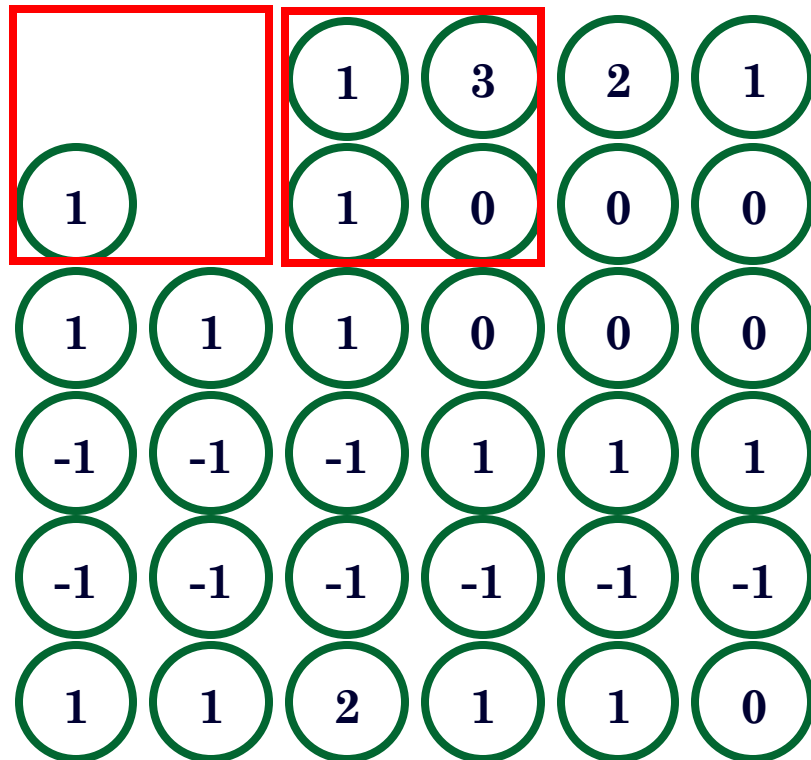
2499499

2×2 pooling

-1	-1	-1
2	2	2
-1	-1	-1

Down-sampling features

-1	2	-1
-1	2	-1
-1	2	-1



How Can We Detect Features at Different Scale?



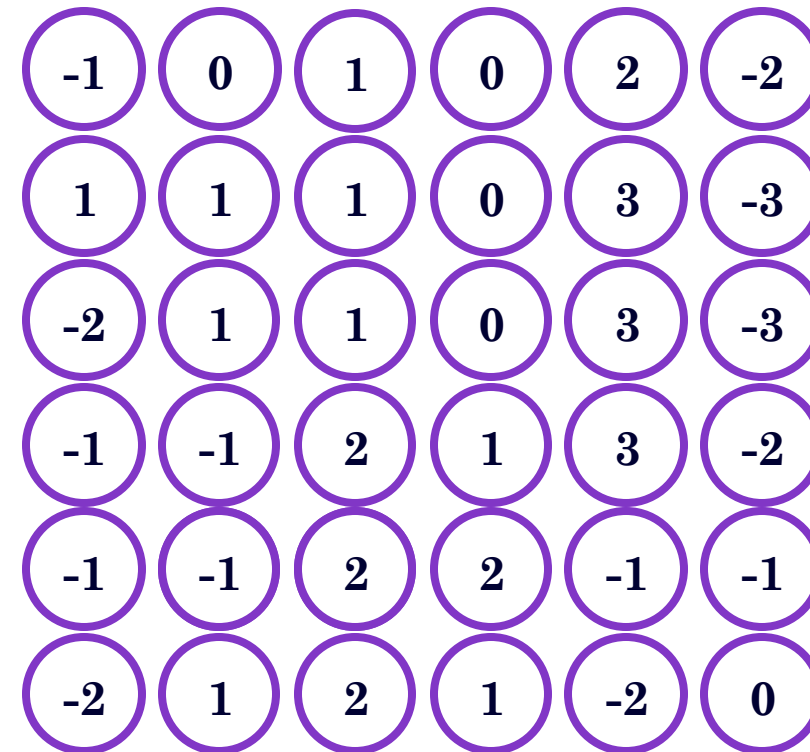
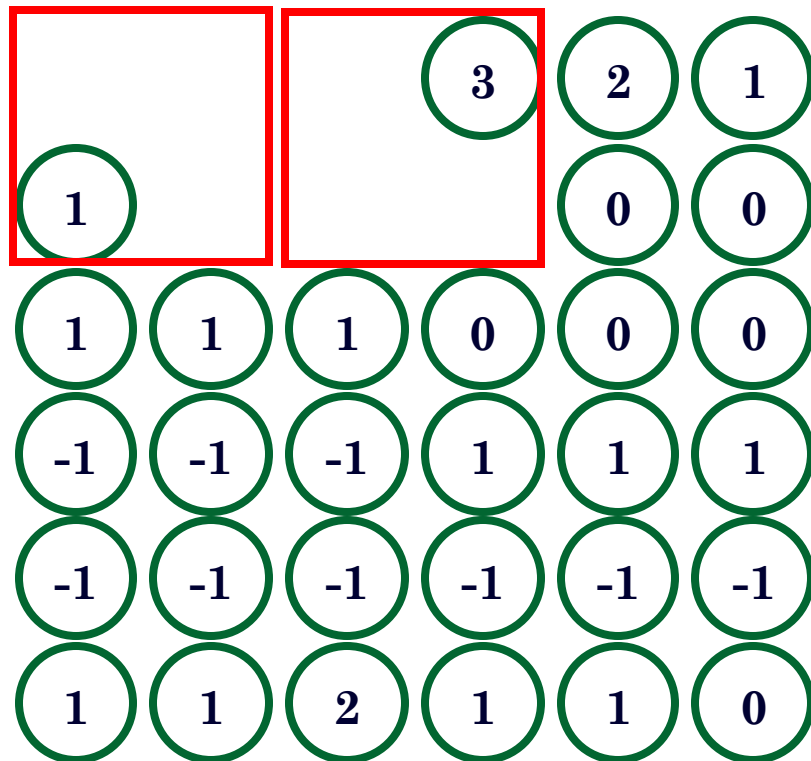
2499499

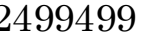
2×2 pooling

-1	-1	-1
2	2	2
-1	-1	-1

Down-sampling features

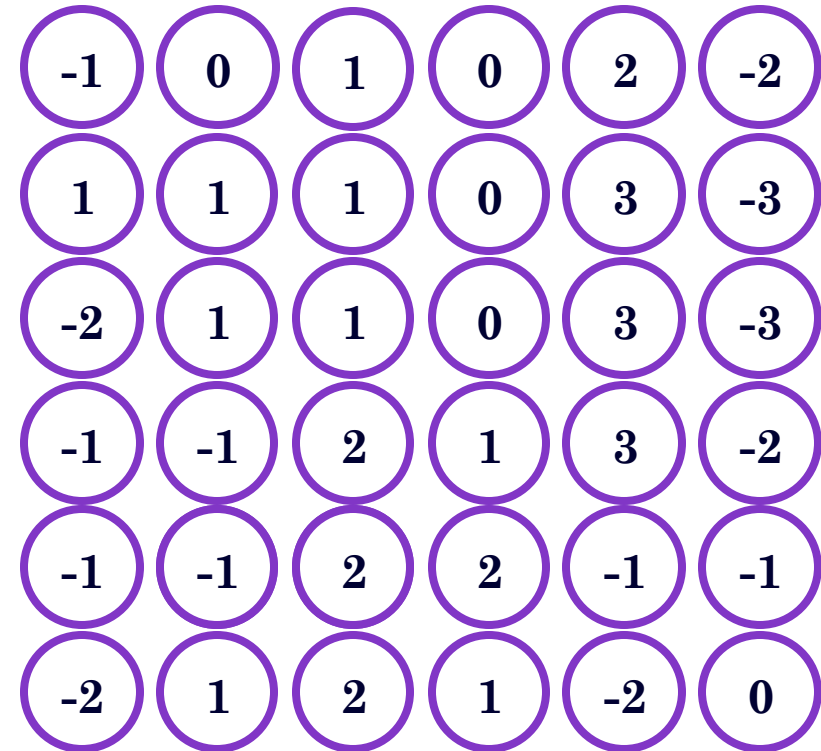
-1	2	-1
-1	2	-1
-1	2	-1





2×2 pooling

Down-sampling features



How Can We Detect Features at Different Scale?



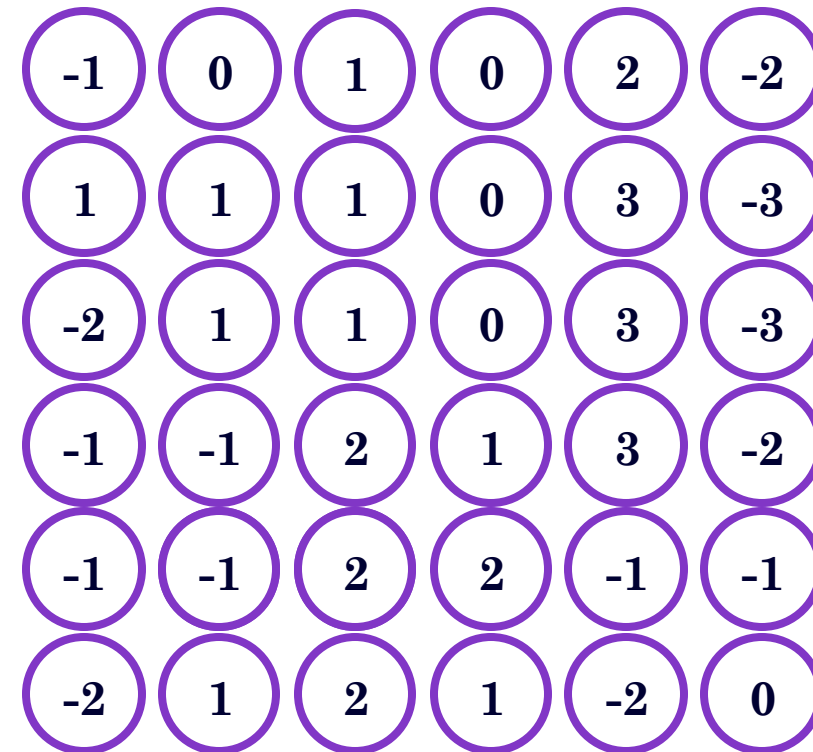
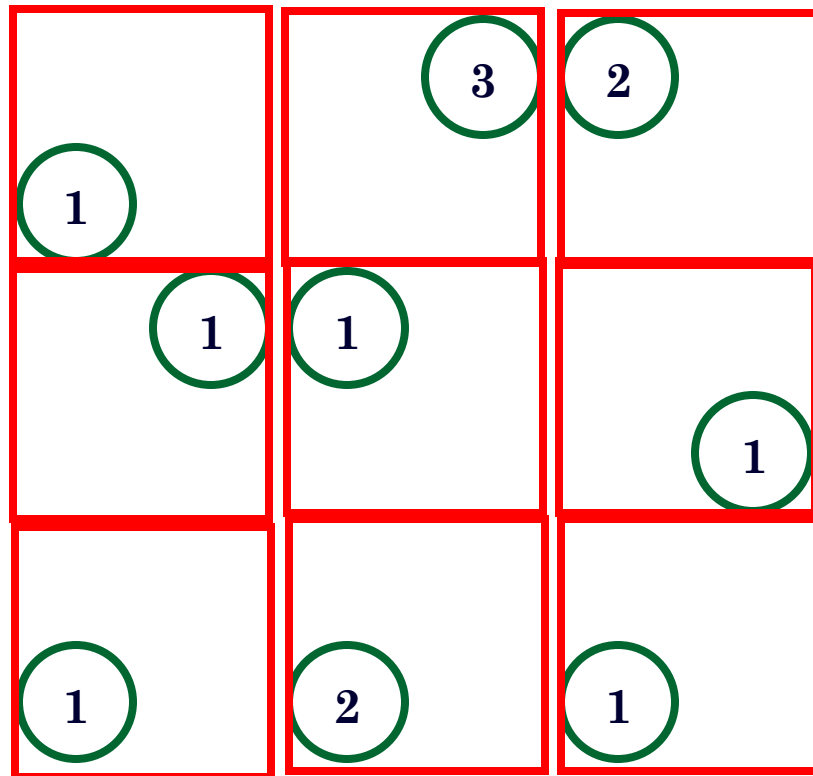
2499499

2×2 pooling

-1	-1	-1
2	2	2
-1	-1	-1

Down-sampling features

-1	2	-1
-1	2	-1
-1	2	-1



How Can We Detect Features at Different Scale?



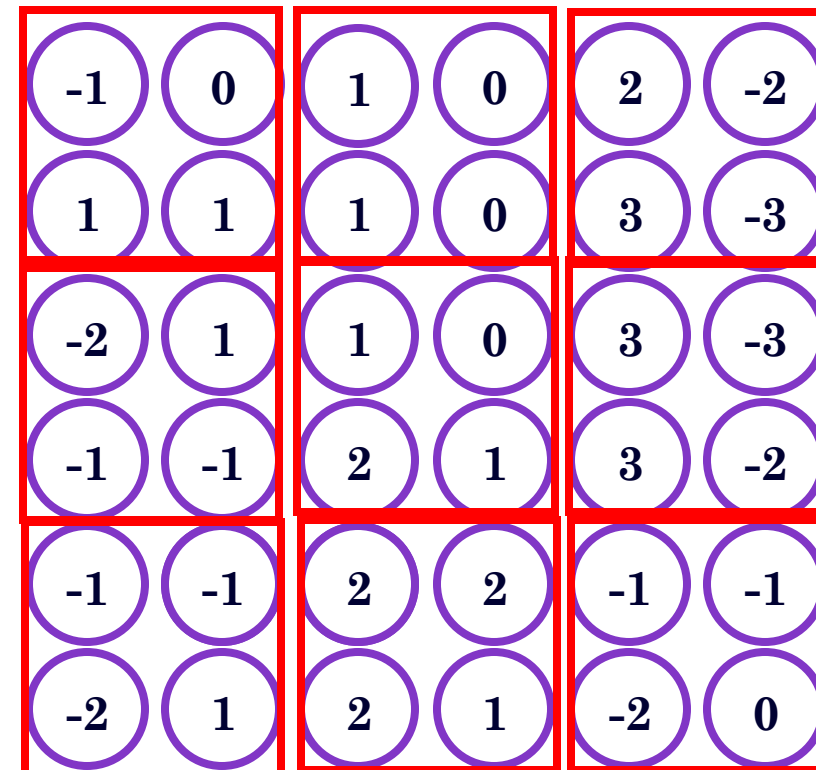
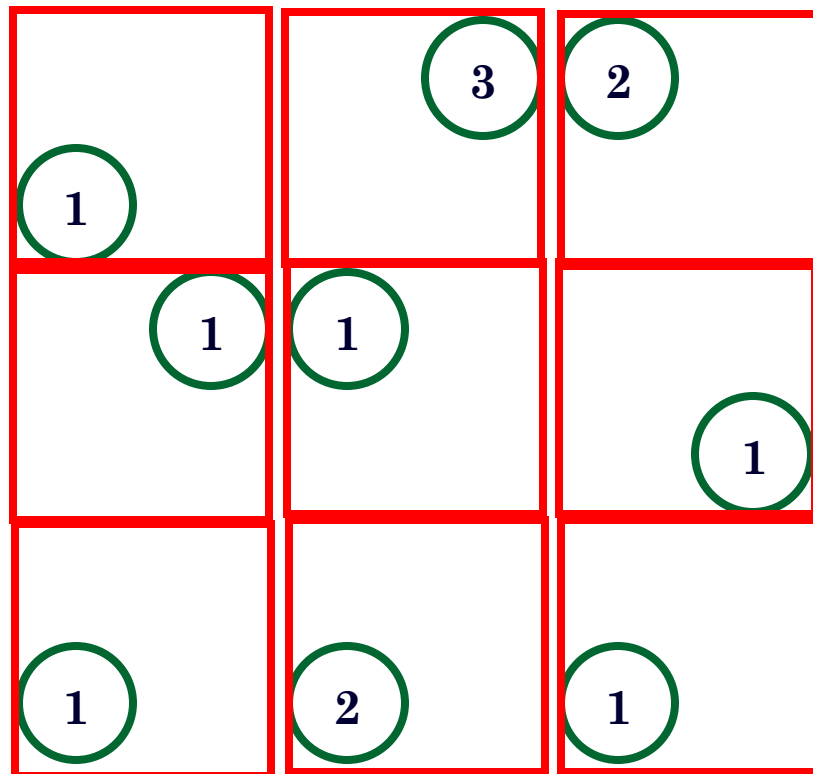
2499499

2×2 pooling

-1	-1	-1
2	2	2
-1	-1	-1

Down-sampling features

-1	2	-1
-1	2	-1
-1	2	-1



How Can We Detect Features at Different Scale?



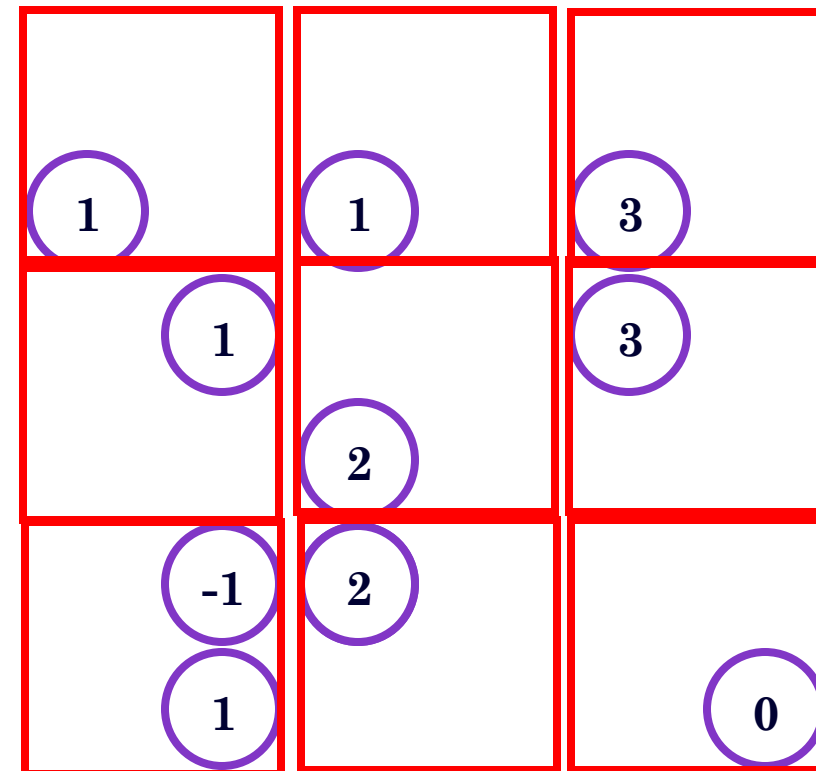
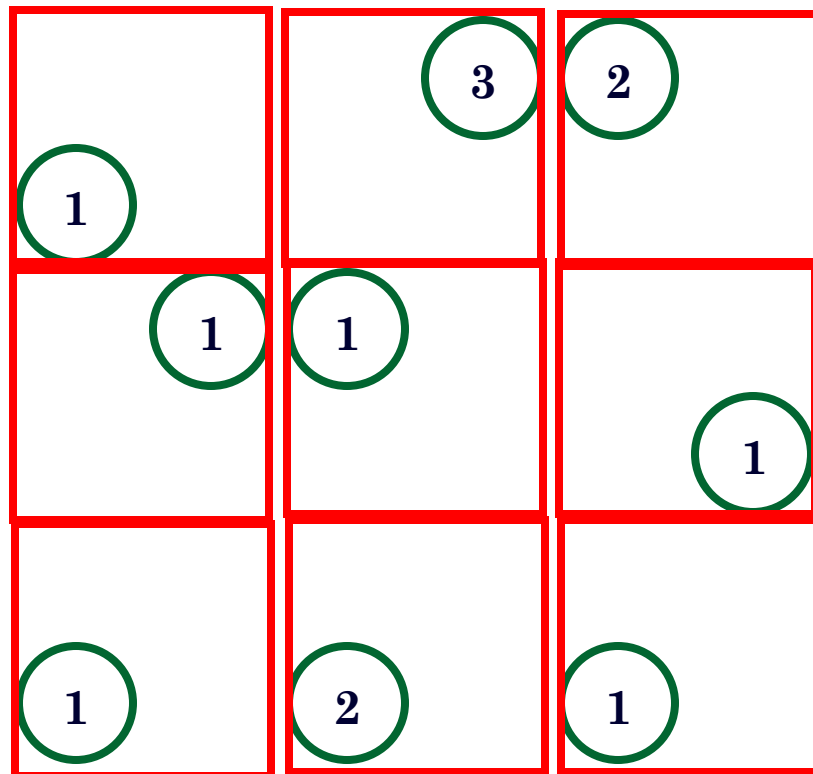
2499499

2×2 pooling

-1	-1	-1
2	2	2
-1	-1	-1

Down-sampling features

-1	2	-1
-1	2	-1
-1	2	-1



How Can We Detect Features at Different Scale?



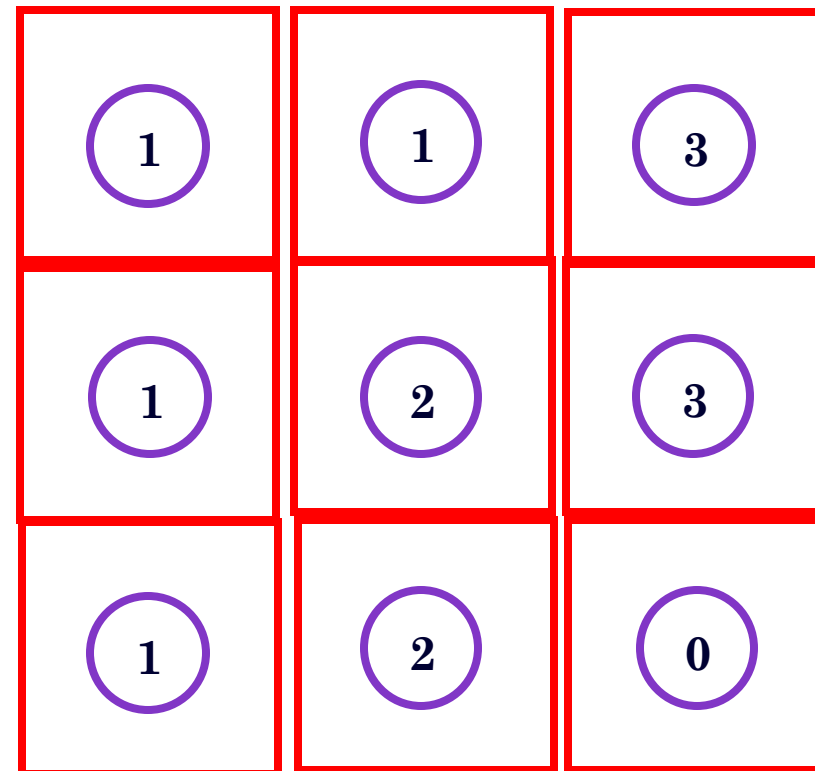
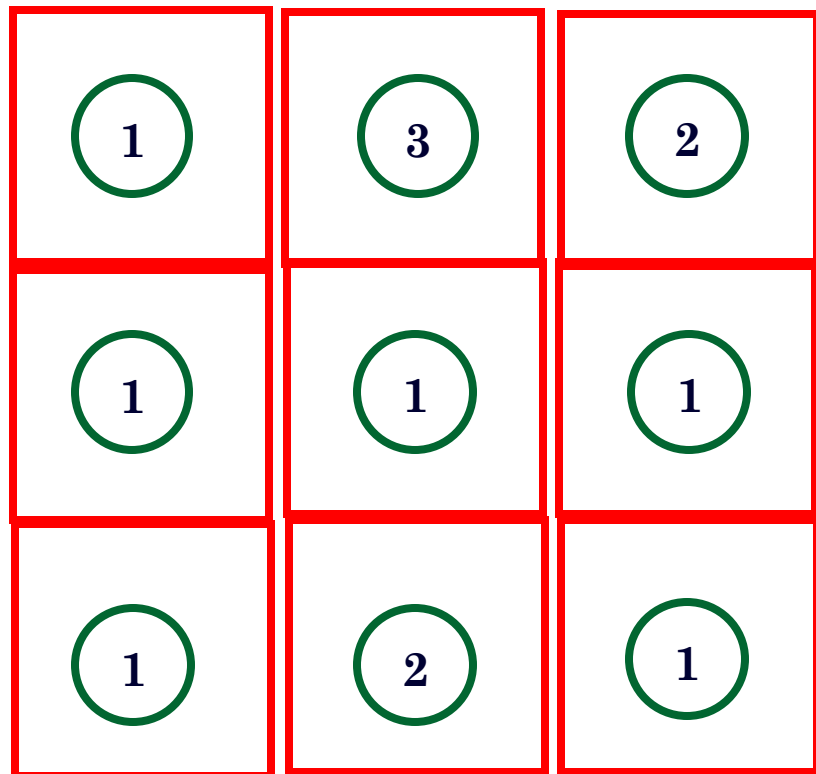
2499499

2×2 pooling

-1	-1	-1
2	2	2
-1	-1	-1

Down-sampling features

-1	2	-1
-1	2	-1
-1	2	-1



How Can We Detect Features at Different Scale?



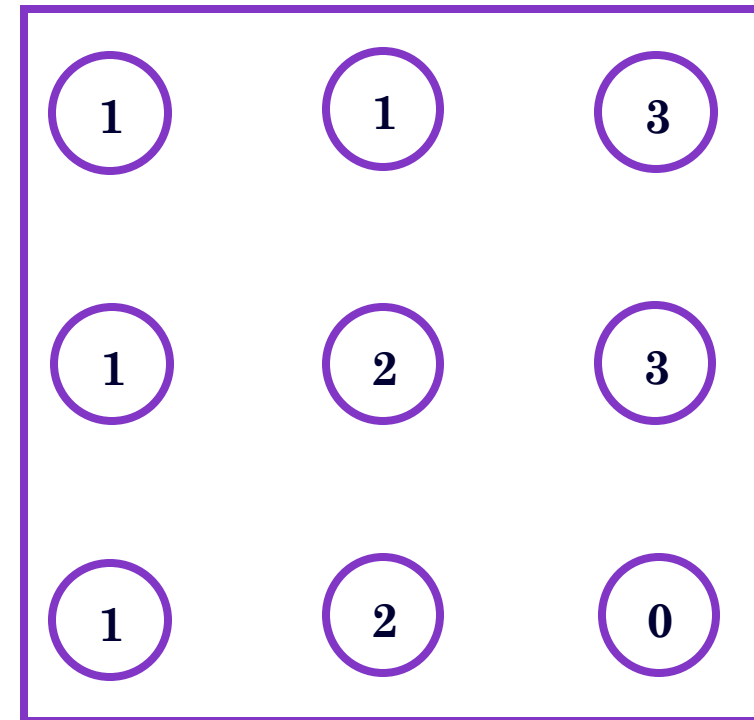
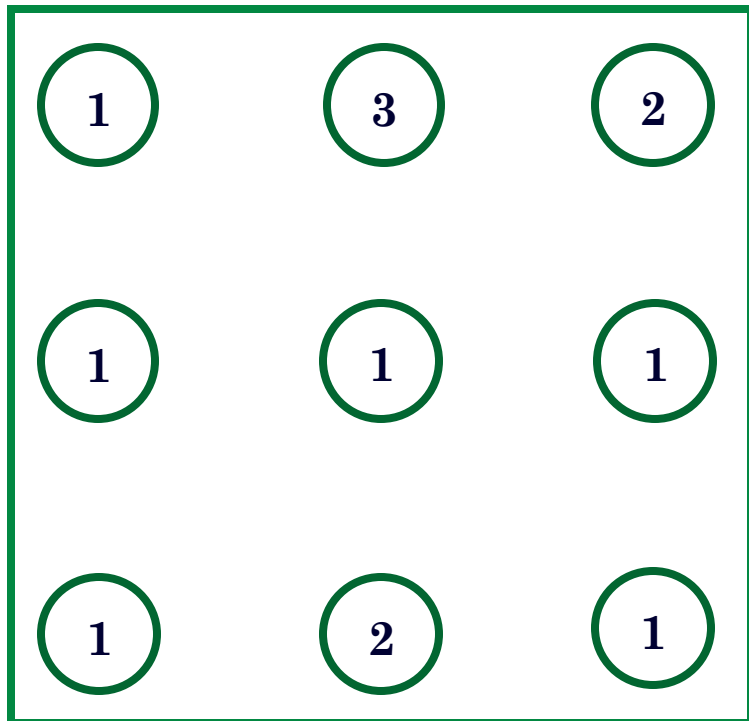
2499499

2×2 pooling

-1	-1	-1
2	2	2
-1	-1	-1

Down-sampling features

-1	2	-1
-1	2	-1
-1	2	-1



CNN Design

- Network Design
Considerations for Images
- Element of the
Convolutional Neural
Network
- CNN Design
- CNN History

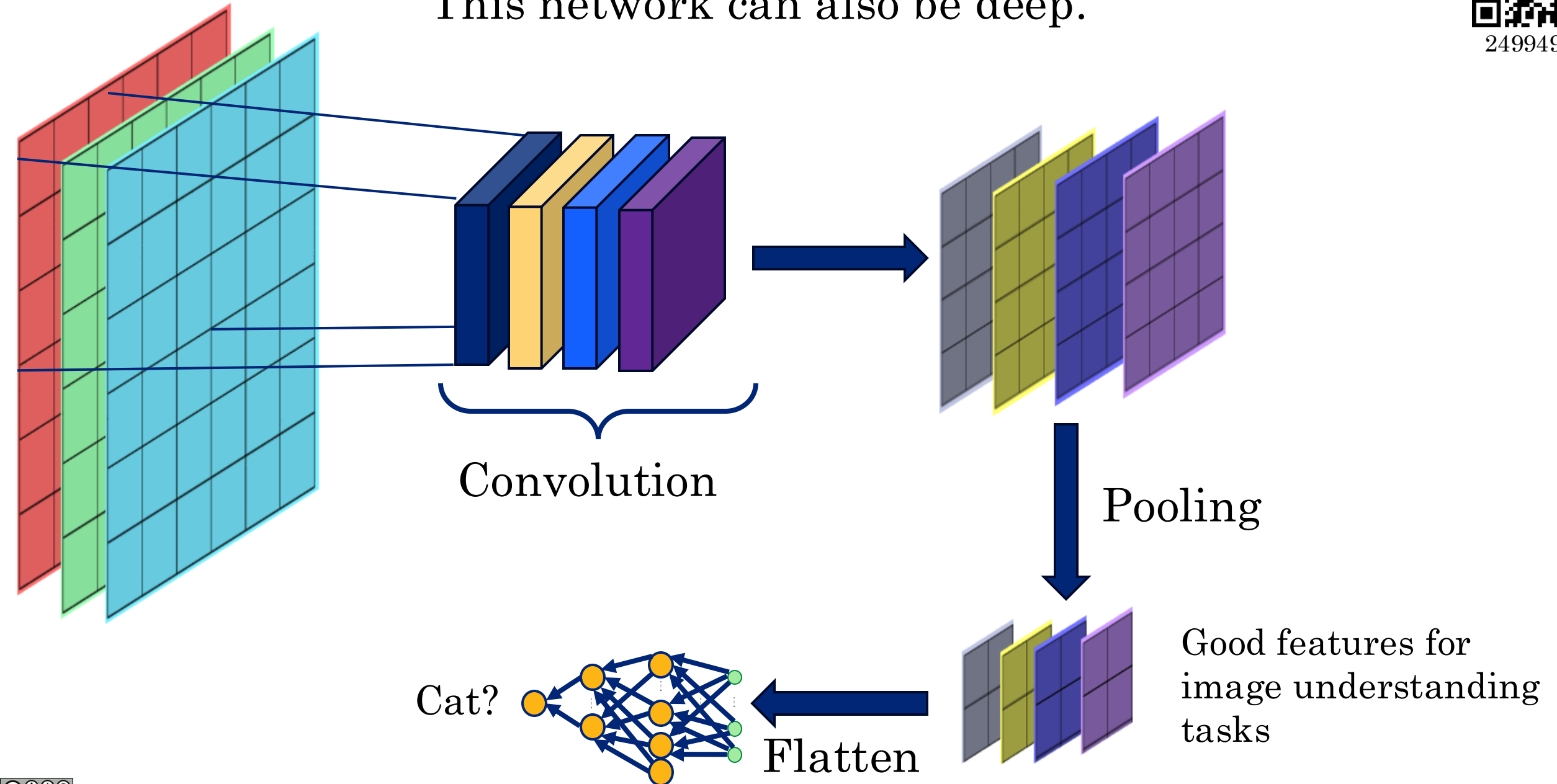


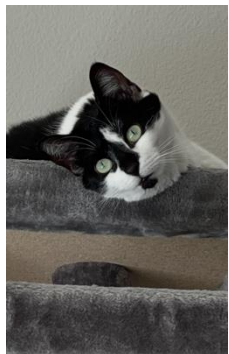
2499499



2499499

This network can also be deep.





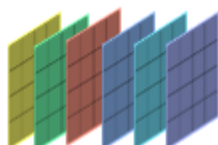
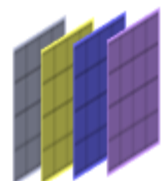
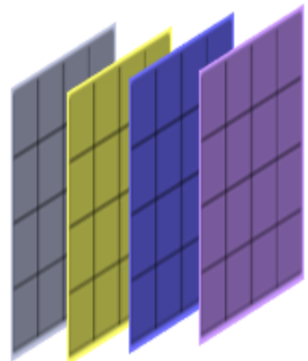
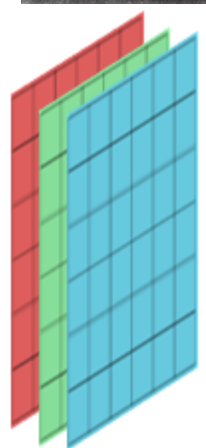
Convolution

Pooling

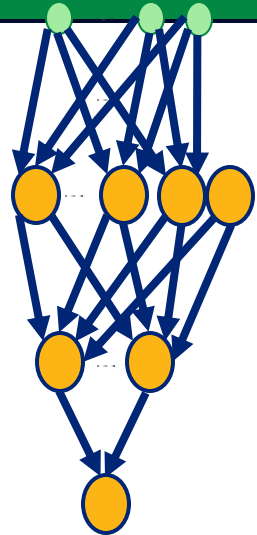


Convolution

Pooling



Flatten



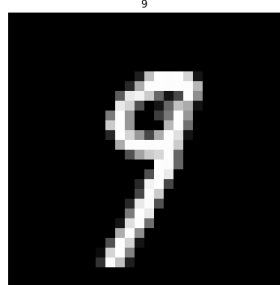
Output



2499499



2499499



$1 \times 28 \times 28$



8 kernels of $1 \times 3 \times 3$

Convolution

Pooling

$8 \times 28 \times 28$



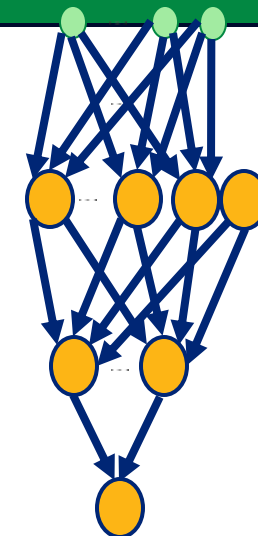
Convolution

Pooling

```
def __init__(self, num_classes=10):  
    super().__init__()   
  
    self.conv1 = nn.Conv2d(3, 8, kernel_size=3, padding=1)
```

```
nn.Conv2d(  
    in_channels,    # # of input feature maps (e.g., 3 for RGB)  
    out_channels,   # # of filters you learn → # of output feature maps  
    kernel_size,    # filter size (int or (kh, kw))  
    stride=1,       # step size of the filter (int or (sh, sw))  
    padding=0,      # add zeros around the image (int, tuple, or 'same')  
    bias=True,      # learn an additive bias per output channel  
    padding_mode='zeros' # 'zeros' (default), 'reflect', 'replicate'  
)
```

Flatten



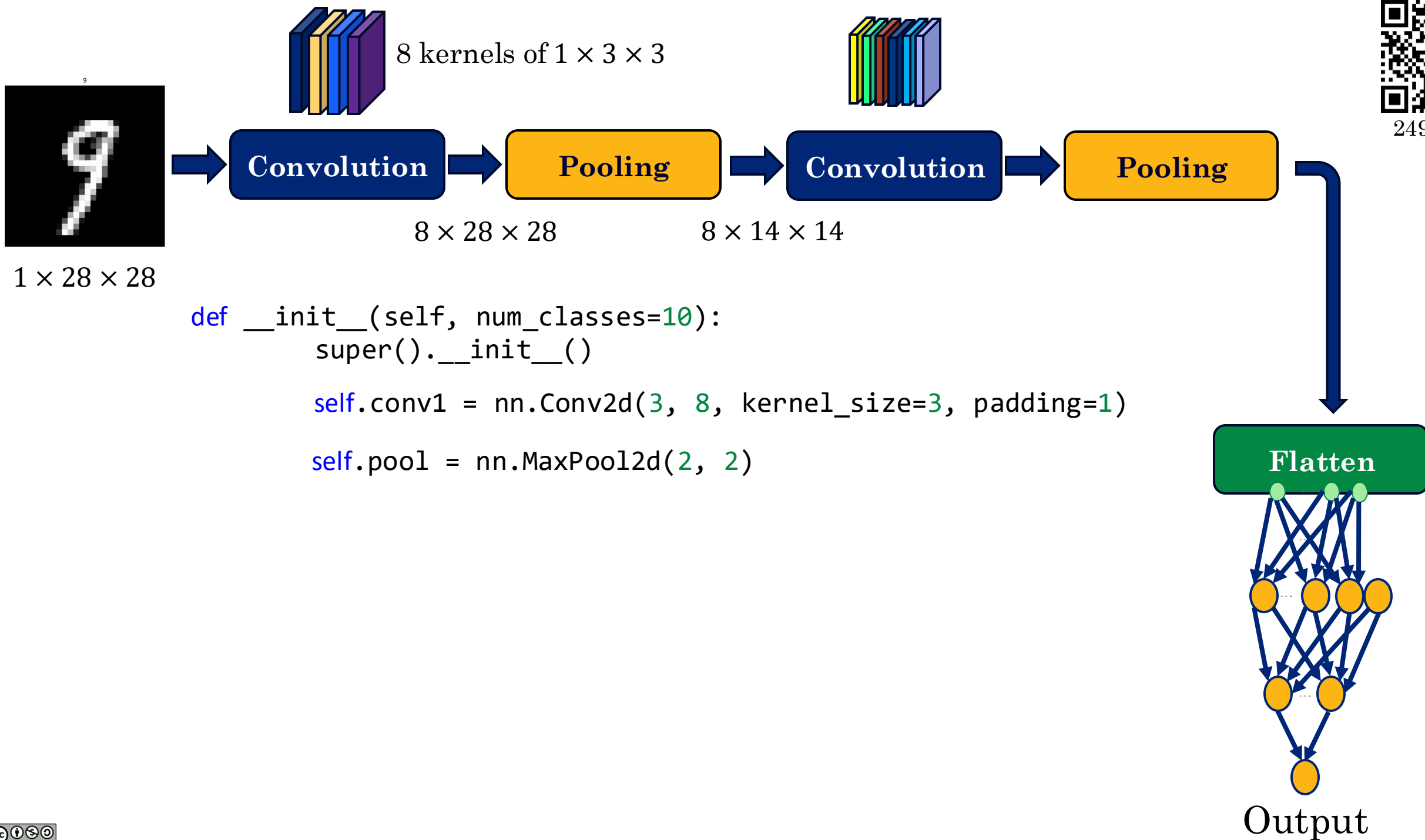
Output

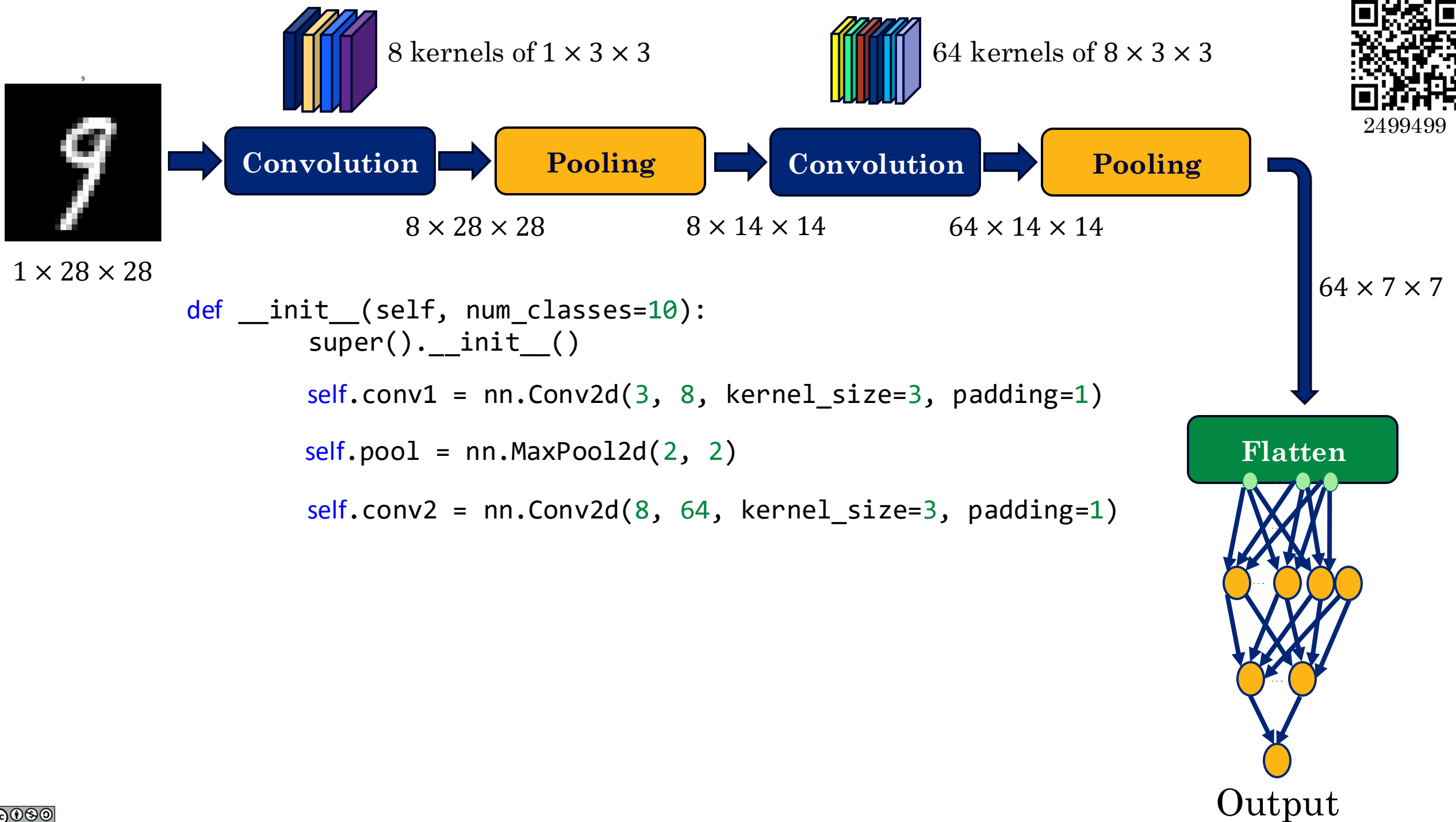


**How many parameters are
trained in the first convolution
layer?**



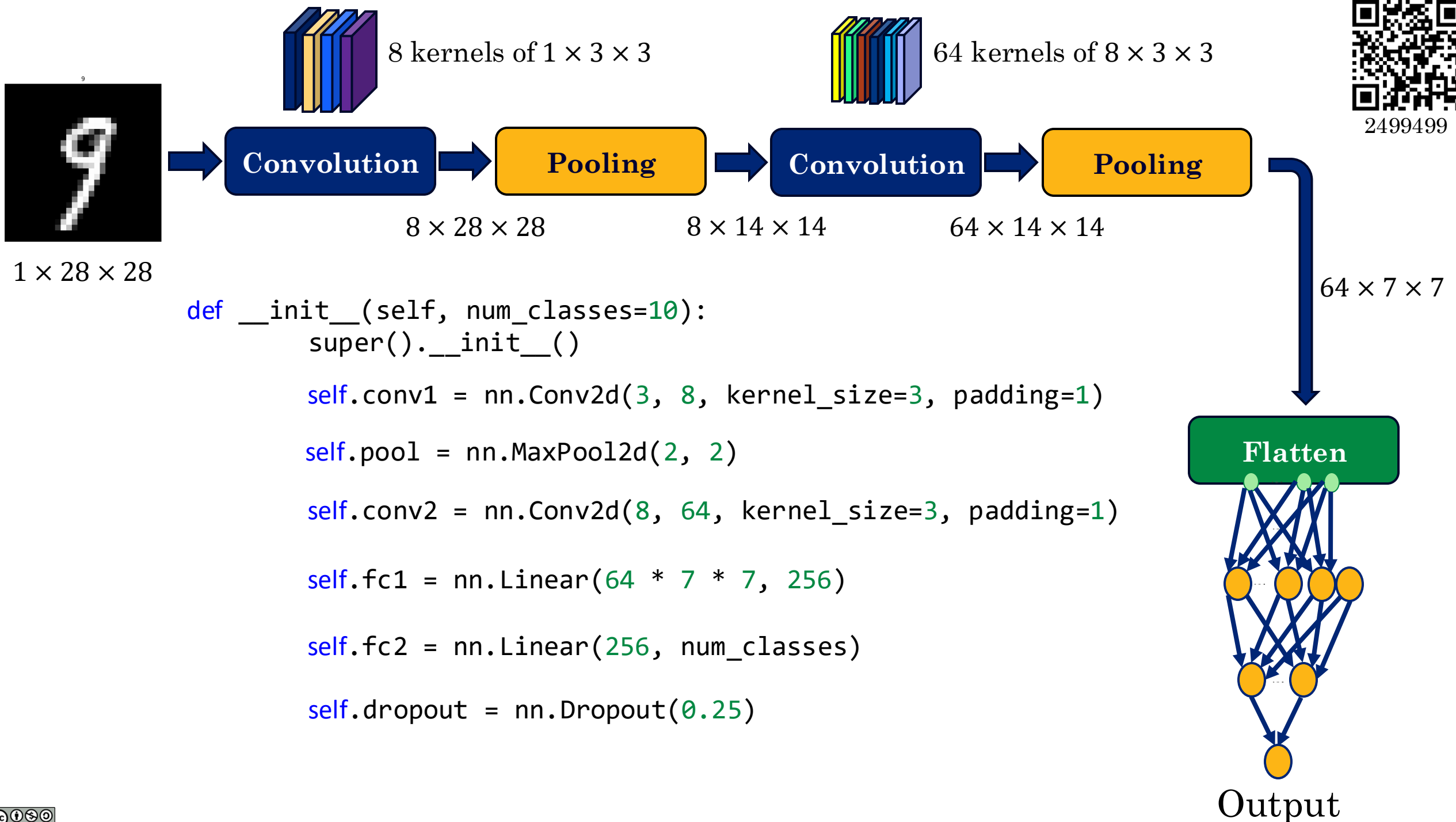
2499499

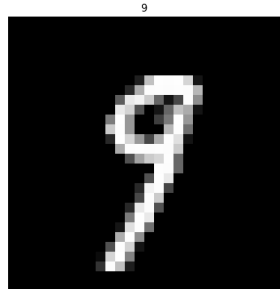






**How many parameters are
trained in the second
convolution layer?**





$1 \times 28 \times 28$



8 kernels of $1 \times 3 \times 3$

Convolution

Pooling

$8 \times 28 \times 28$

$8 \times 14 \times 14$



64 kernels of $8 \times 3 \times 3$

Convolution

Pooling

$64 \times 14 \times 14$

$64 \times 7 \times 7$

```
def forward(self, x):
```

```
    x = self.pool(F.relu(self.conv1(x)))    # 8x14x14
```

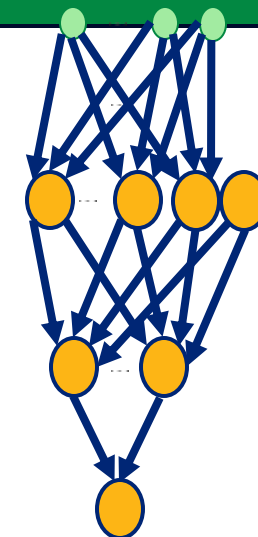
```
    x = self.pool(F.relu(self.conv2(x)))    # 64x7x7
```

```
    x = x.view(x.size(0), -1)                # flatten: 64*7*7
```

```
    x = self.dropout(F.relu(self.fc1(x)))
```

```
    return self.fc2(x)
```

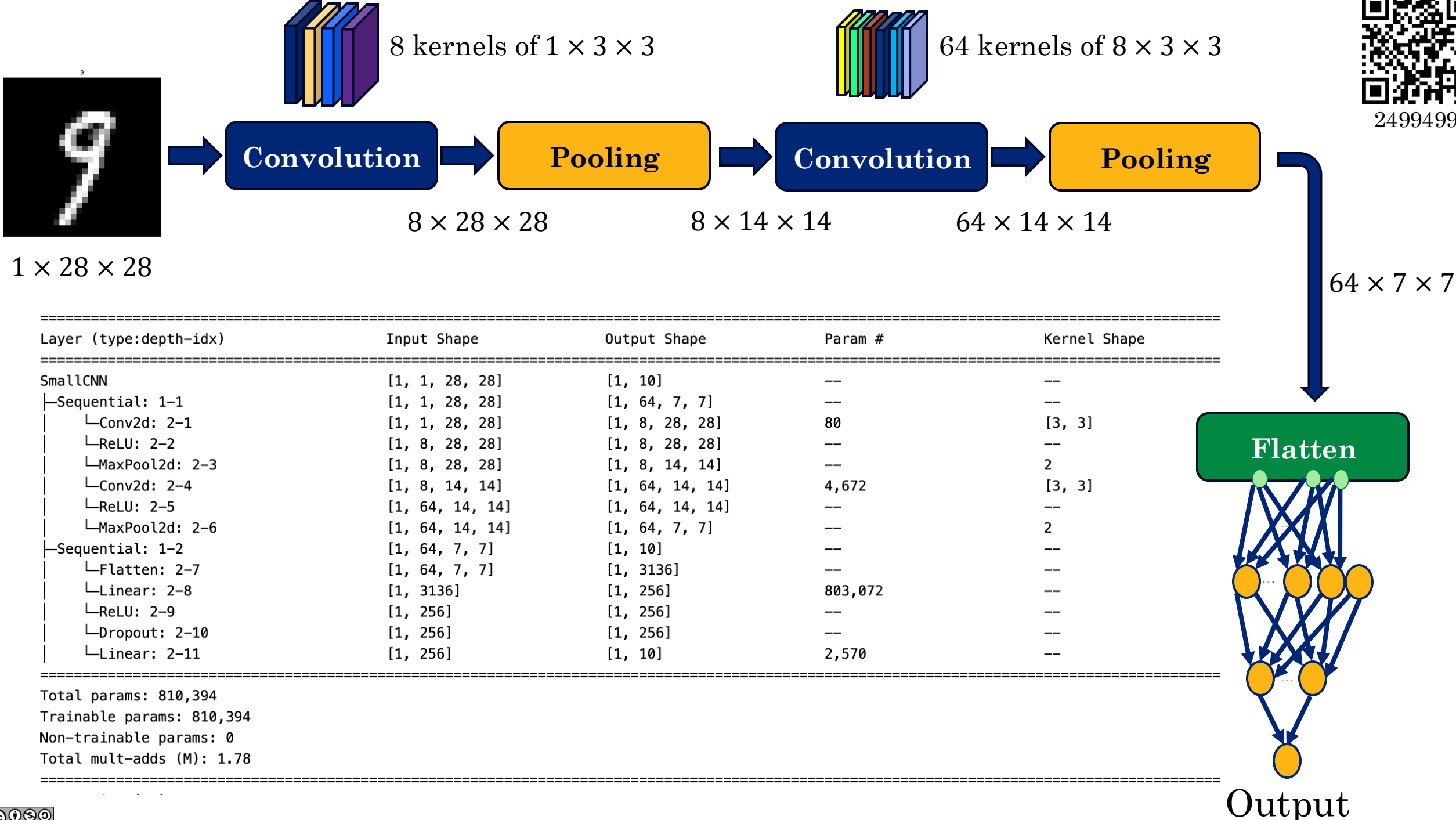
Flatten

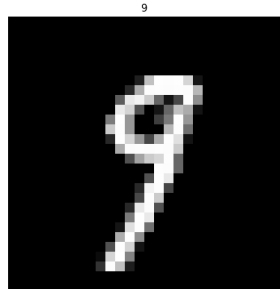


Output



2499499





$1 \times 28 \times 28$



8 kernels of $1 \times 3 \times 3$

Convolution

$8 \times 28 \times 28$

Pooling

$8 \times 14 \times 14$



64 kernels of $8 \times 3 \times 3$

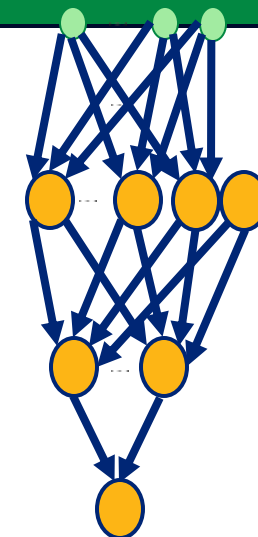
Convolution

$64 \times 14 \times 14$

Pooling

$64 \times 7 \times 7$

Flatten



Output

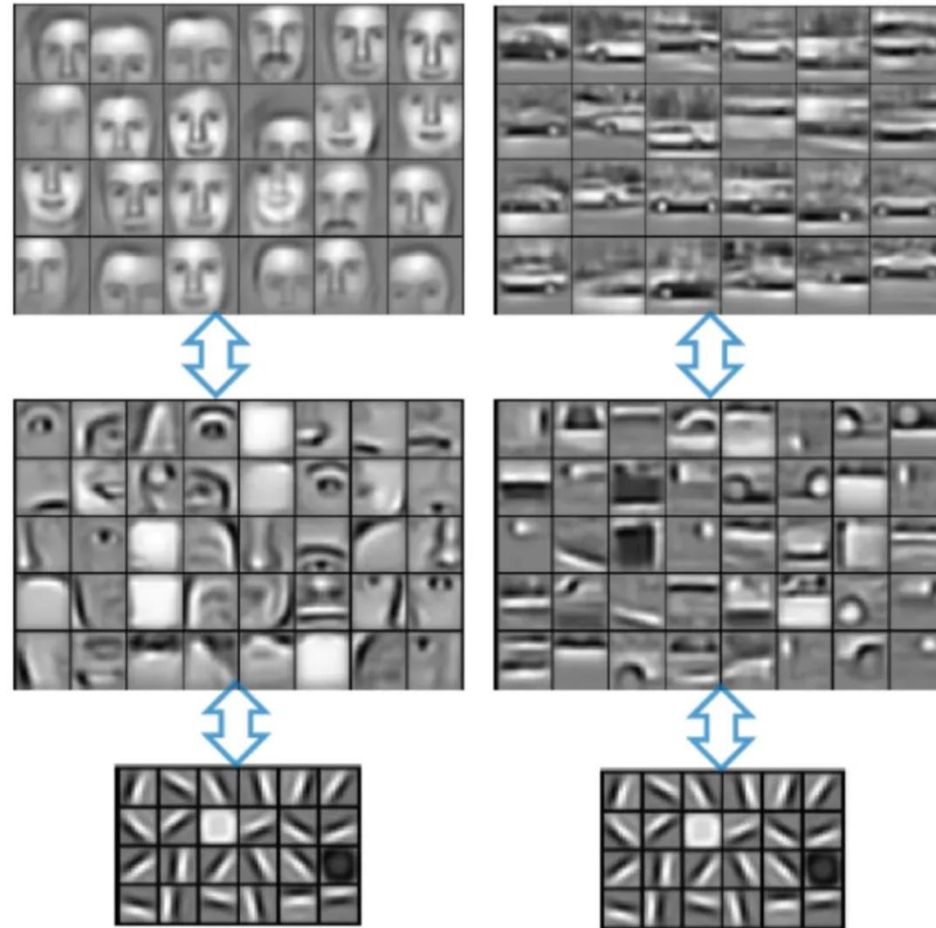
```
class SmallCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(1, 8, 3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Conv2d(8, 64, 3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2), )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(64*7*7, 256),
            nn.ReLU(),
            nn.Dropout(0.25),
            nn.Linear(256, 10))

    def forward(self, x):
        x = self.features(x)
        return self.classifier(x)
```



2499499

What Do the Filters Learn?



CNN History

- Network Design
Considerations for Images
- Element of the
Convolutional Neural
Network
- CNN Design
- CNN History

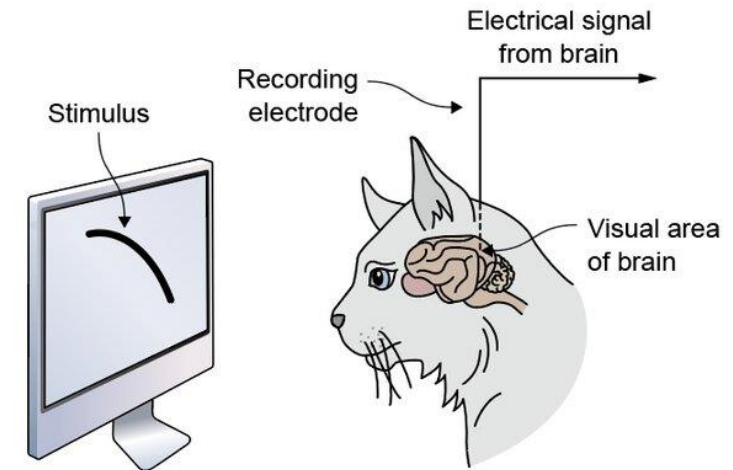


2499499

History



- How do animals see?
 - What is the neural process from eye to recognition?
- Hubel and Wiesel 1959
 - First study on neural correlates of vision.
 - Receptive Fields in Cat's visual cortex
 - 24 cats, anaesthetized, immobilized, on artificial respirators
 - Electrodes in brain
 - Beamed light of different patterns into the eyes and measured neural responses



Gabor Filters



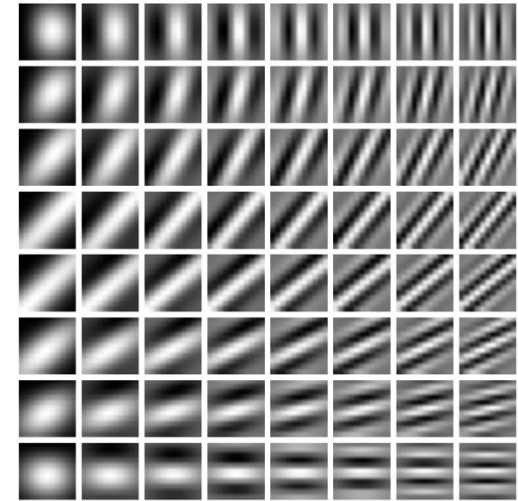
- Hubel and Wiesel study introduced two types of cells in brain:
 - **Simple cells:** Strong response to visual inputs with a simple edge oriented at a particular angle and located at a particular position.
 - More studies showed that the responses from simple cells can be modeled as Gabor filters:

$$G(x, y) = A \exp(-\alpha \tilde{x}^2 - \beta \tilde{y}^2) \sin(\omega \tilde{x} + \phi)$$

Decay envelope to localize the filter around (x_0, y_0)

$$\begin{aligned}\tilde{x} &= (x - x_0) \cos \theta + (y - y_0) \sin \theta \\ \tilde{y} &= -(x - x_0) \sin \theta + (y - y_0) \cos \theta\end{aligned}$$

- **Complex cells:** Respond to more complex stimuli derived by combining and processing the output of simple cells.



1980s and 1990s ...



- By the end of the 1980's, several papers were produced that considerably advanced the field.
- The idea of backpropagation was first published in French by Yann LeCun in 1985.
- November 1998, LeCun published one of his most recognized papers describing a “modern” CNN architecture for document recognition, called LeNet-5.

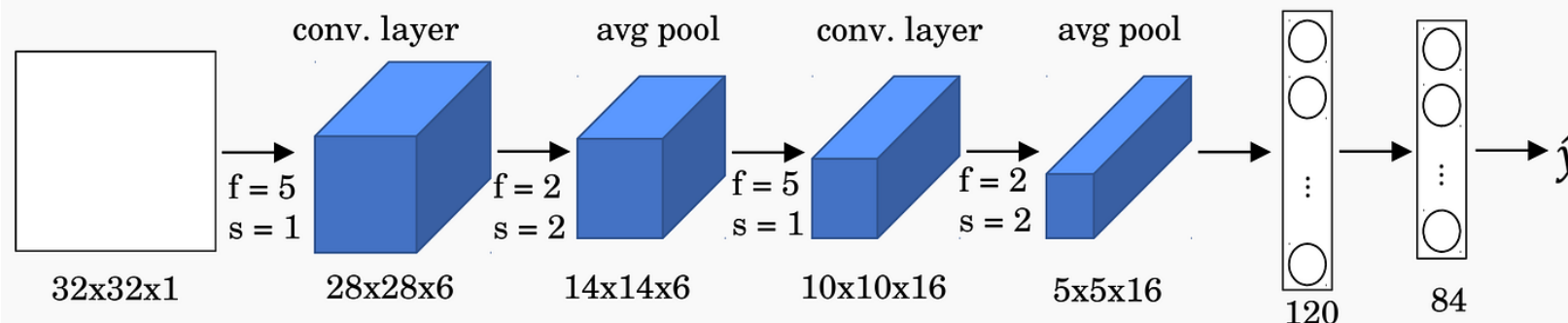


Image from
<https://medium.com/>

LeCun, Yann, et al. “Gradient-based learning applied to document recognition.”
Proceedings of the IEEE 86.11 (1998): 2278–2324.

The ImageNet Task (2009)



- 14 million images in more than 20,000 hierarchical categories with hand-annotation
 - Indicate what objects are pictured
 - In at least one million of the images, bounding boxes are also provided





ImageNet Large-Scale Visual Recognition Challenge (ILSVRC) (2010)

- ILSVRC evaluates algorithms for object detection and image classification at large scale.
- **The classification task:**
 - Get the “correct” class in your top 5 bets. There are 1000 classes.
- **The localization task:**
 - For each bet, put a box around the object. Your box must have at least 50% overlap with the correct box.

Examples from the Test Set



cheetah

cheetah

leopard

snow leopard

Egyptian cat



bullet train is like a plane, with in-train magazine and a jacket that you can plug your headphones into and listen to

bullet train

bullet train

passenger car

subway train

electric locomotive



hand glass

scissors

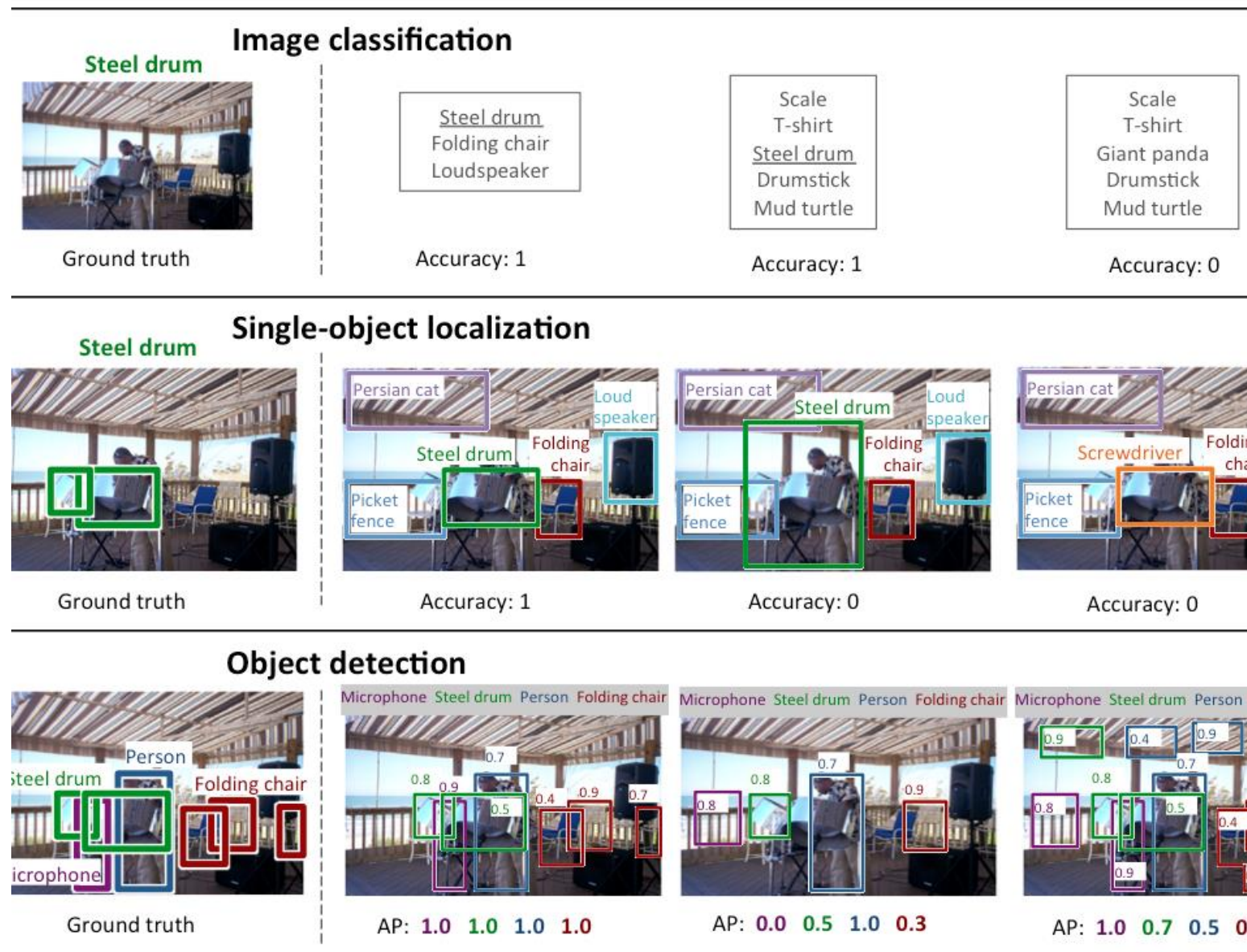
hand glass

frying pan

stethoscope

ImageNet Large-Scale Visual Recognition Challenge (ILSVRC)

The first column shows the ground truth labeling on an example image, and the next three show three sample outputs with the corresponding evaluation score.



Credit: <https://arxiv.org/html/1409.0575>

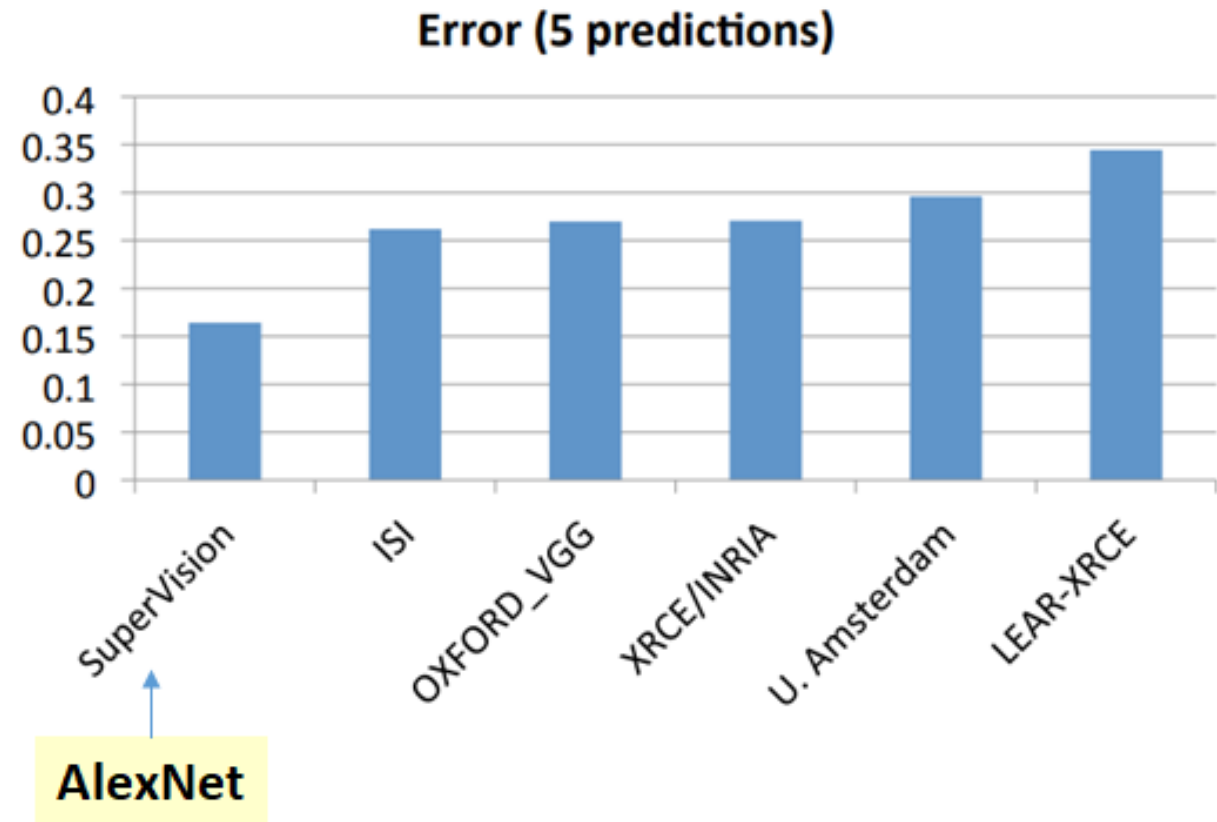


2499499

The ILSVRC-2012 Competition on ImageNet

Some of the best existing computer vision methods were tried on this dataset by leading computer vision groups from Oxford, INRIA, XRCE

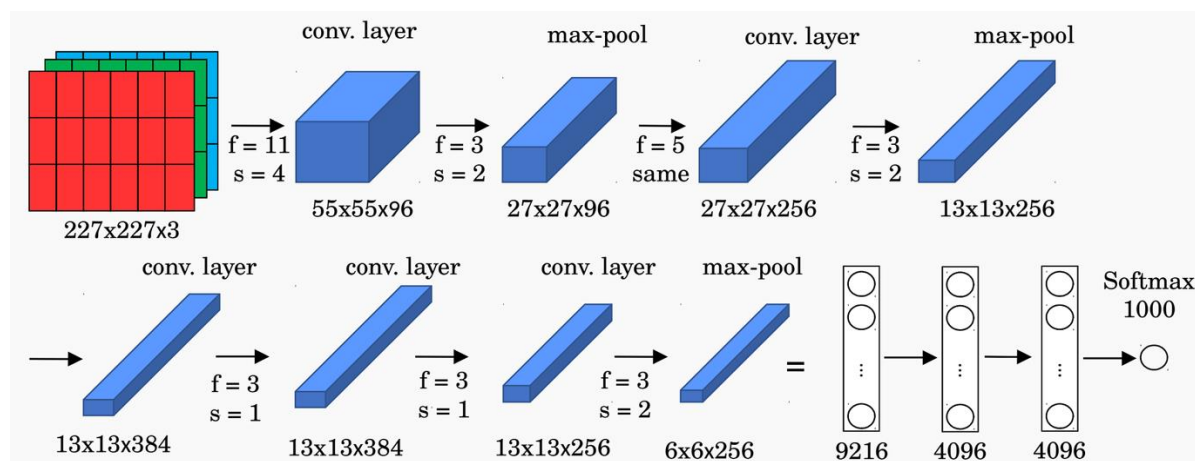
Ranking of the best results from each team



AlexNet ([Link](#))



2499499



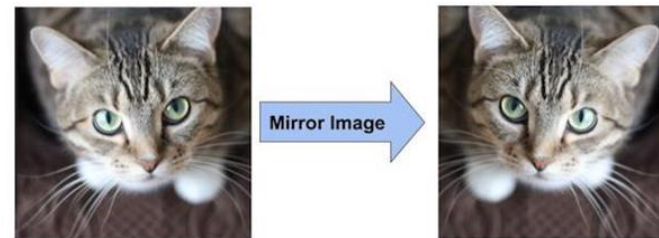
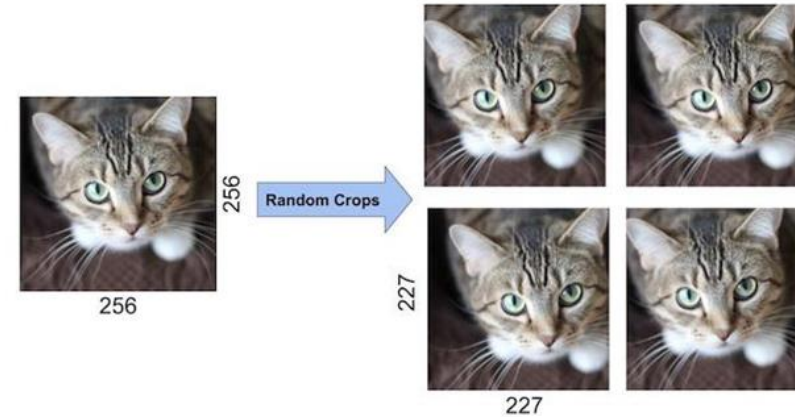
Credit: <https://medium.com/data-science/advanced-topics-in-deep-convolutional-neural-networks-71ef1190522d>

- Alex Krizhevsky (NeurIPS 2012)'s AlexNet architecture is shown here.
- The paper has 185,000 citations.
- ReLU activation is used in every hidden layer. These train much faster and are more expressive than logistic units.

AlexNet's Tricks to Improve Generalization



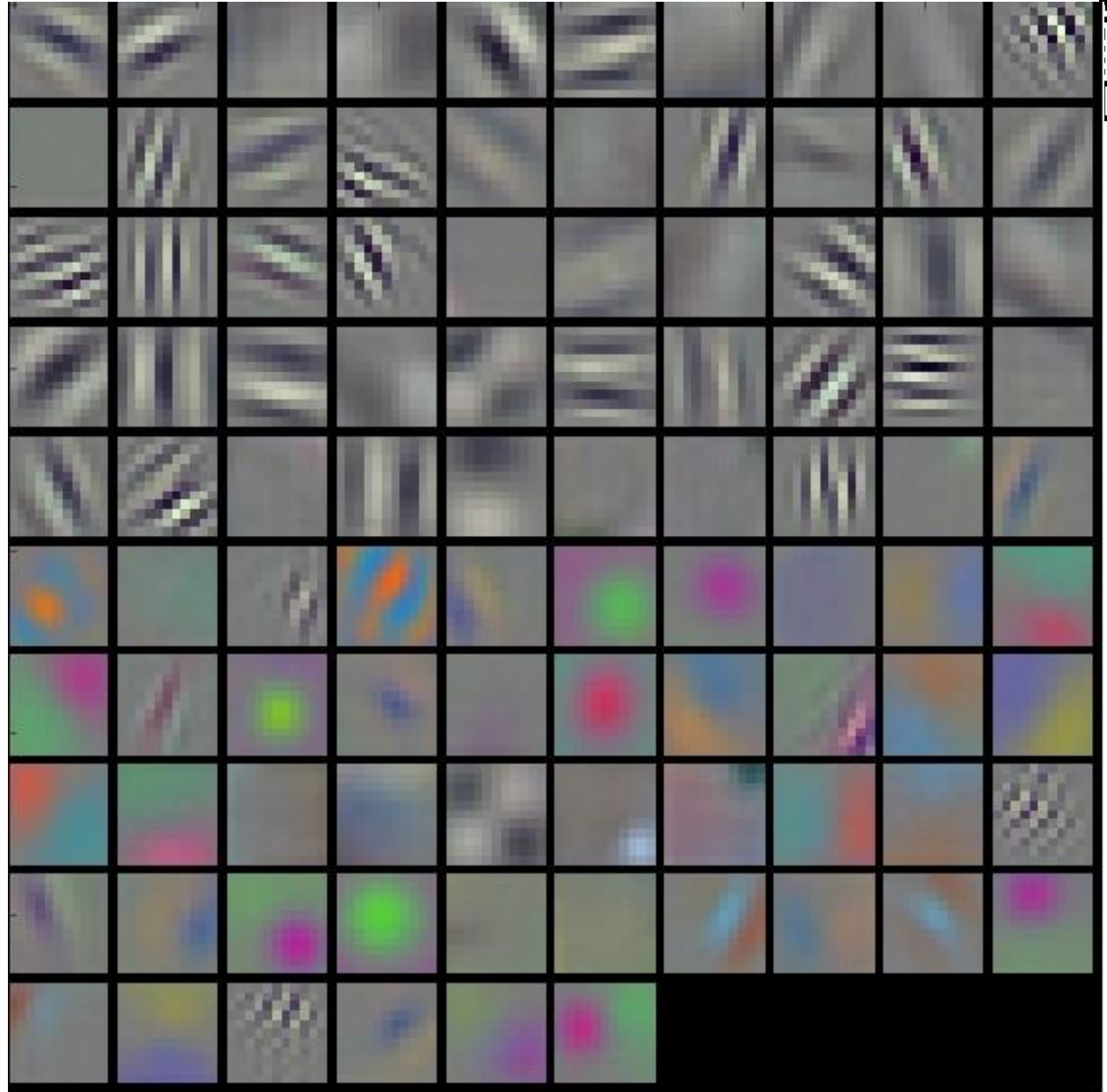
- Train on random 224×224 patches from the 256×256 images to get more data. Also use left-right.
 - At test time, combine the opinions from ten different patches: The four 224×224 corner patches plus the central 224×224 patch plus the reflections of those five.



AlexNet's First Convolution Layer

96 convolutional kernels of size $11 \times 11 \times 3$ learned by the first convolutional layer on the $224 \times 224 \times 3$ input images.

The top 48 kernels were learned on GPU 1 while the bottom 48 kernels were learned on GPU 2.



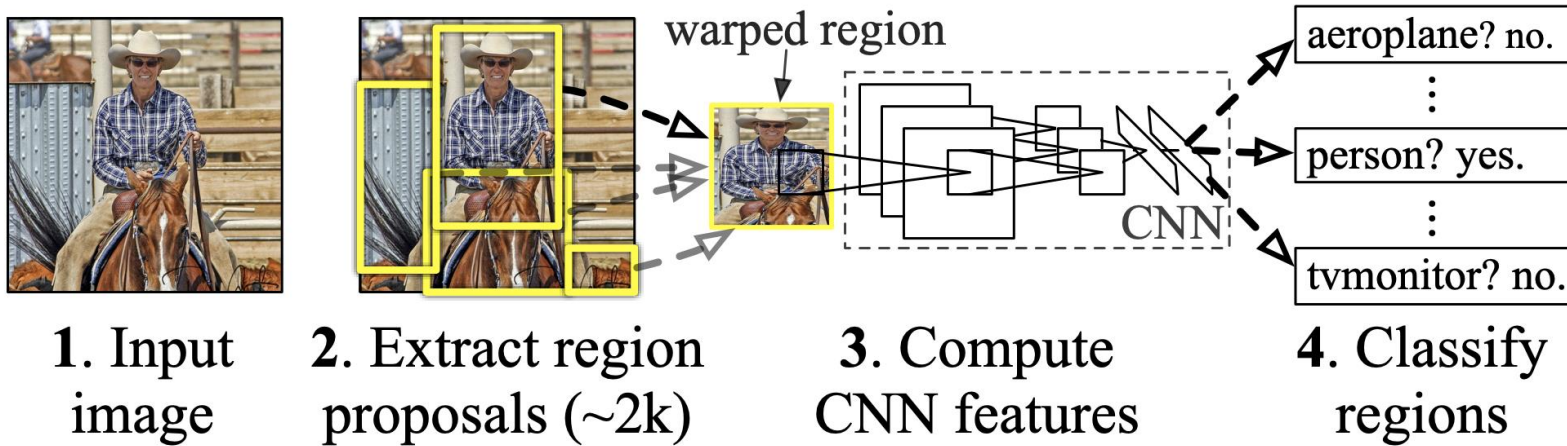
2499499

How About Higher Layers?



- Which images make a specific neuron activate?

R-CNN: *Regions with CNN features*



Ross Girshick, Jeff Donahue, Trevor Darrell, Jitendra Malik, "Rich feature hierarchies for accurate object detection semantic segmentation", CVPR, 2014

How About Higher Layers?

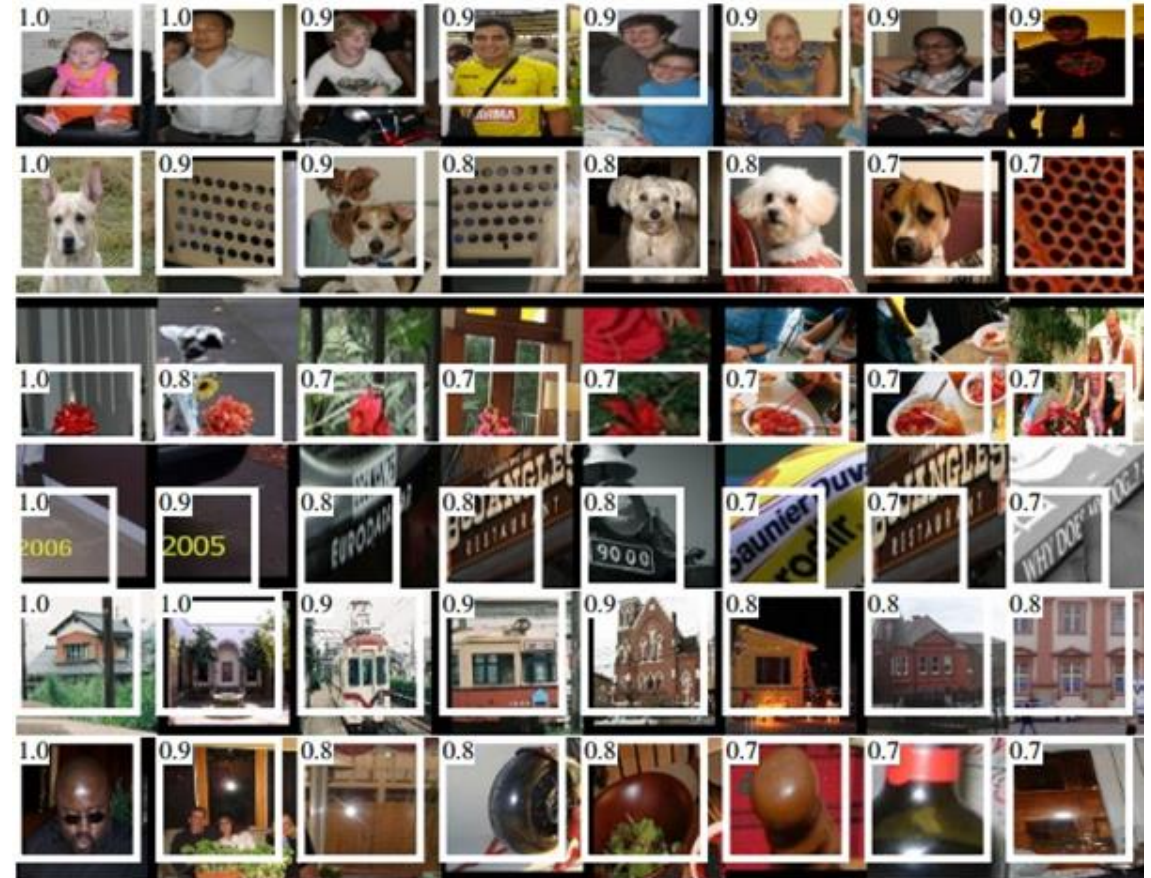


- Which images make a specific neuron activate?

Top regions for six POOL5 units:

Maximally activating images for some POOL5 (5th pool layer) neurons of an AlexNet. The activation values and the receptive field of the particular neuron are shown in white. Some neurons are responsive to upper bodies, text, or specular highlights.

Ross Girshick, Jeff Donahue, Trevor Darrell, Jitendra Malik, “Rich feature hierarchies for accurate object detection and semantic segmentation”, CVPR, 2014



Lecture 18

Convolutional Neural Network

Reading: *Chapter 10* in Bishop Deep Learning Textbook